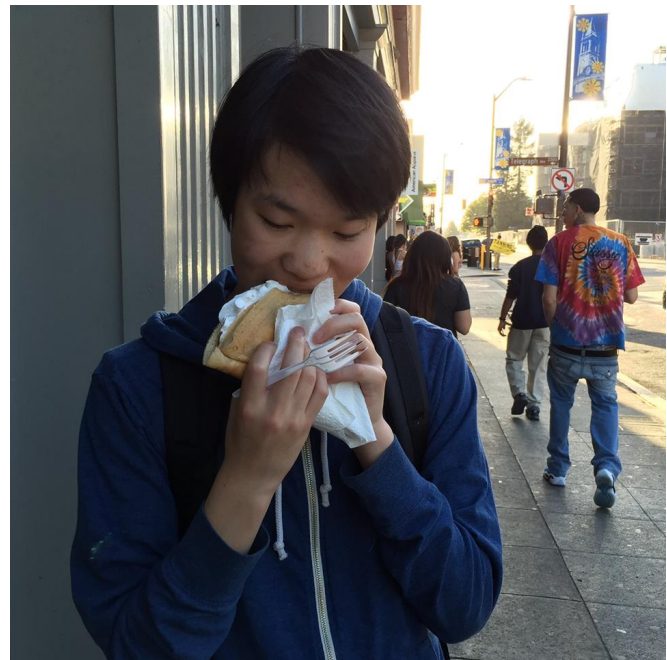


Neural Kernels Without Tangents

Vaishaal Shankar



Alex
Fang



Wenshuo
Guo



Sara
Fridovich-Keil



Ludwig
Schmidt



Jonathan
Ragan-Kelley



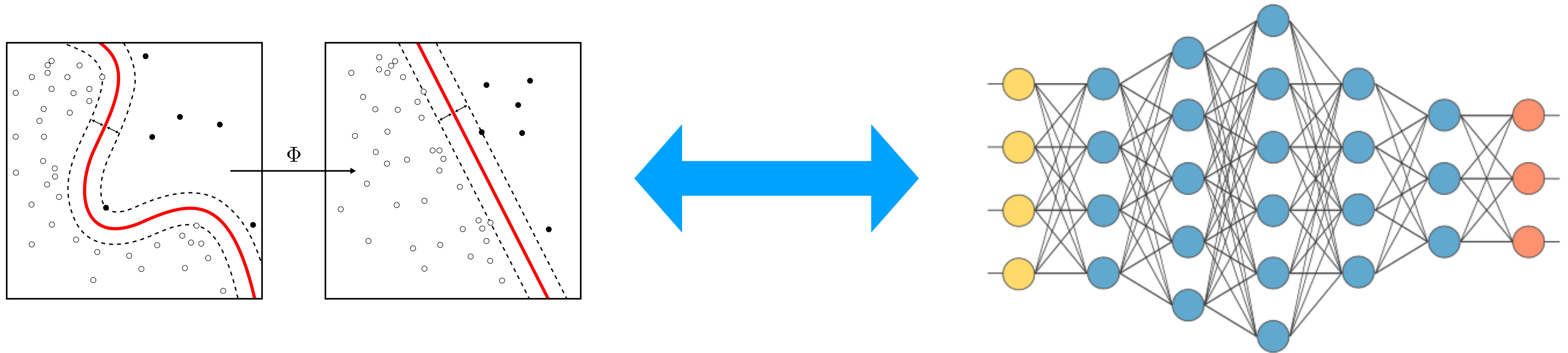
Ben
Recht



Berkeley
UNIVERSITY OF CALIFORNIA



Why Study Kernels?



Recent connections between neural networks and kernel methods

Why don't existing kernels suffice?

RBF kernel on pixels



Frog



Truck

RBF Kernel Value = 0.629

Permute pixels with **identical** permutation map.

RBF Kernel value **does not** change.

Prior kernels

- **Explicitly** model neural network gradient flow dynamics to derive an exact kernel (Neural Tangent Kernel) [\[Jacot et al, Arora et al\]](#)
 - Difficult to compute or even write down
 - Low test accuracy **(82% on Cifar-10)**
- **Approximate** iterated convolutional kernel maps [\[Mairal et al\]](#)
 - Often large approximation dimension necessary for best performance **(86% on Cifar-10)**
 - Large approximation dimension precludes deep kernel maps

Our kernels

- Compute deep convolutional kernels **exactly**
- Simple kernel constructions built using analogues of
 - Convolution
 - Average Pooling
 - ReLU
- State of the art kernel test accuracy on Cifar-10 **(90%)** & Cifar-100 **(68%)**
- Kernel performance correlates with neural network performance

An Image Kernel

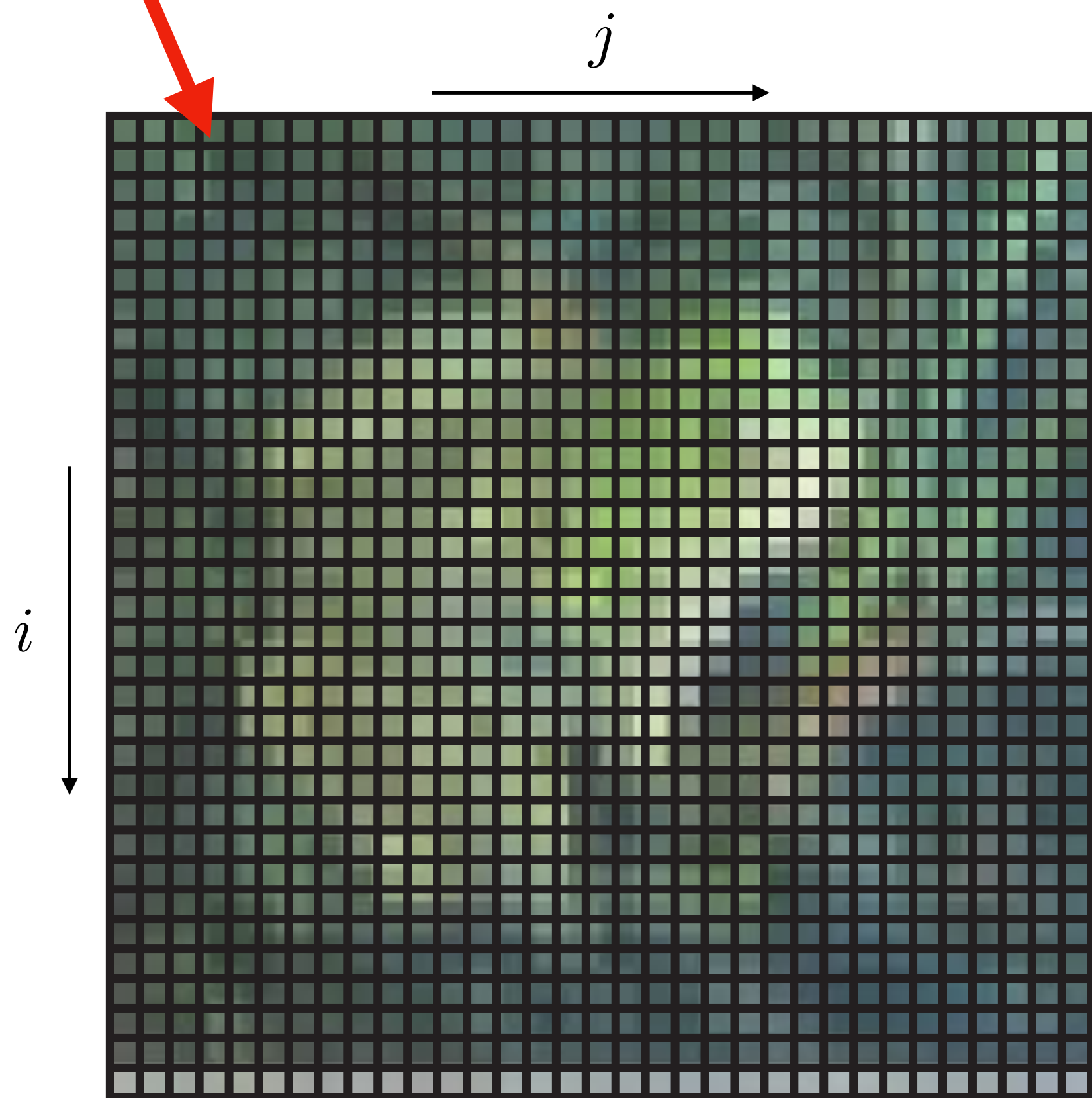
$$K(\text{img } x, \text{img } z) = ?$$

Idea: Compare Sub-Regions

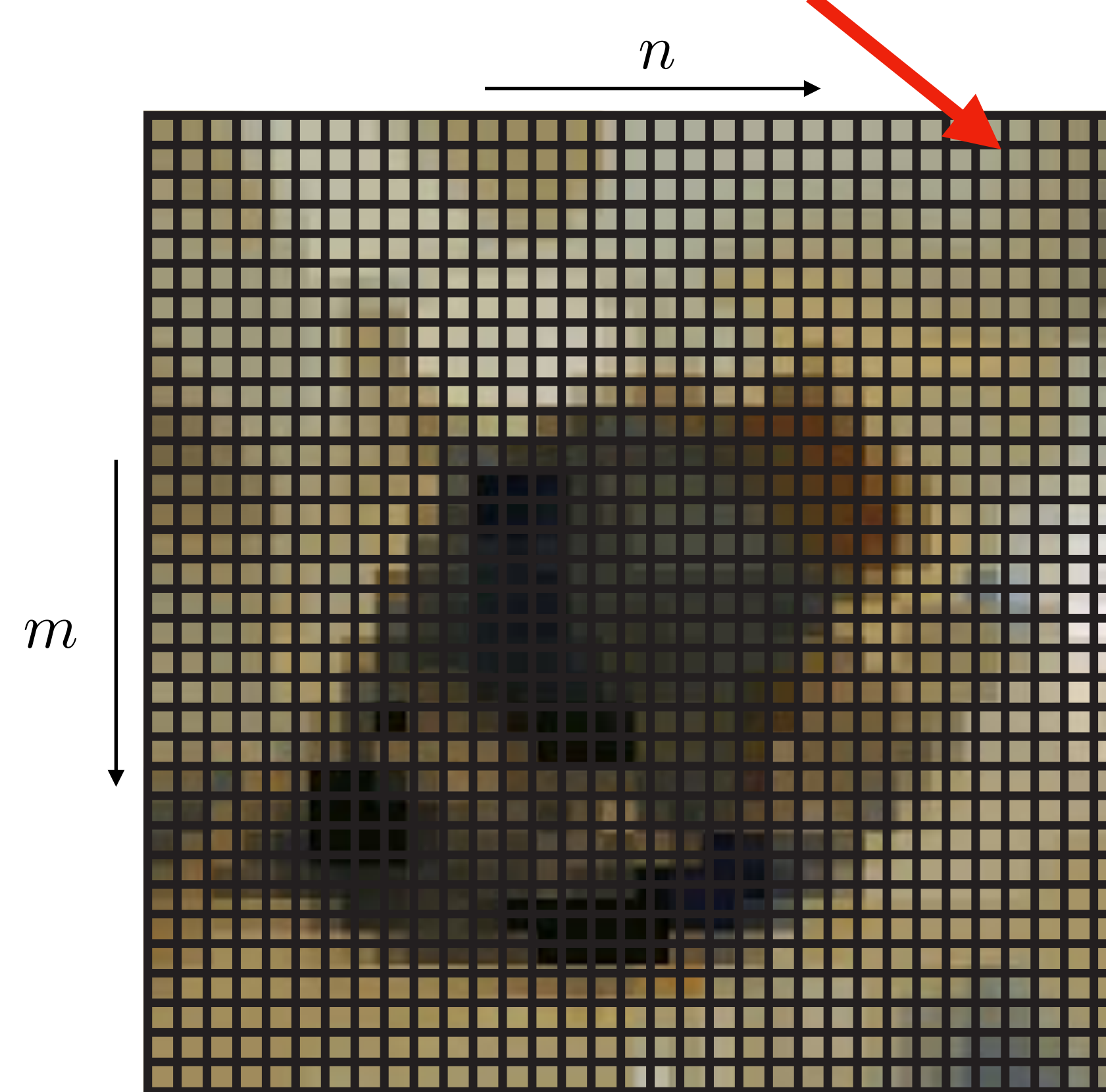
$$x_{ij} \in \mathbb{R}^3$$

Box of vectors.

$$z_{mn} \in \mathbb{R}^3$$



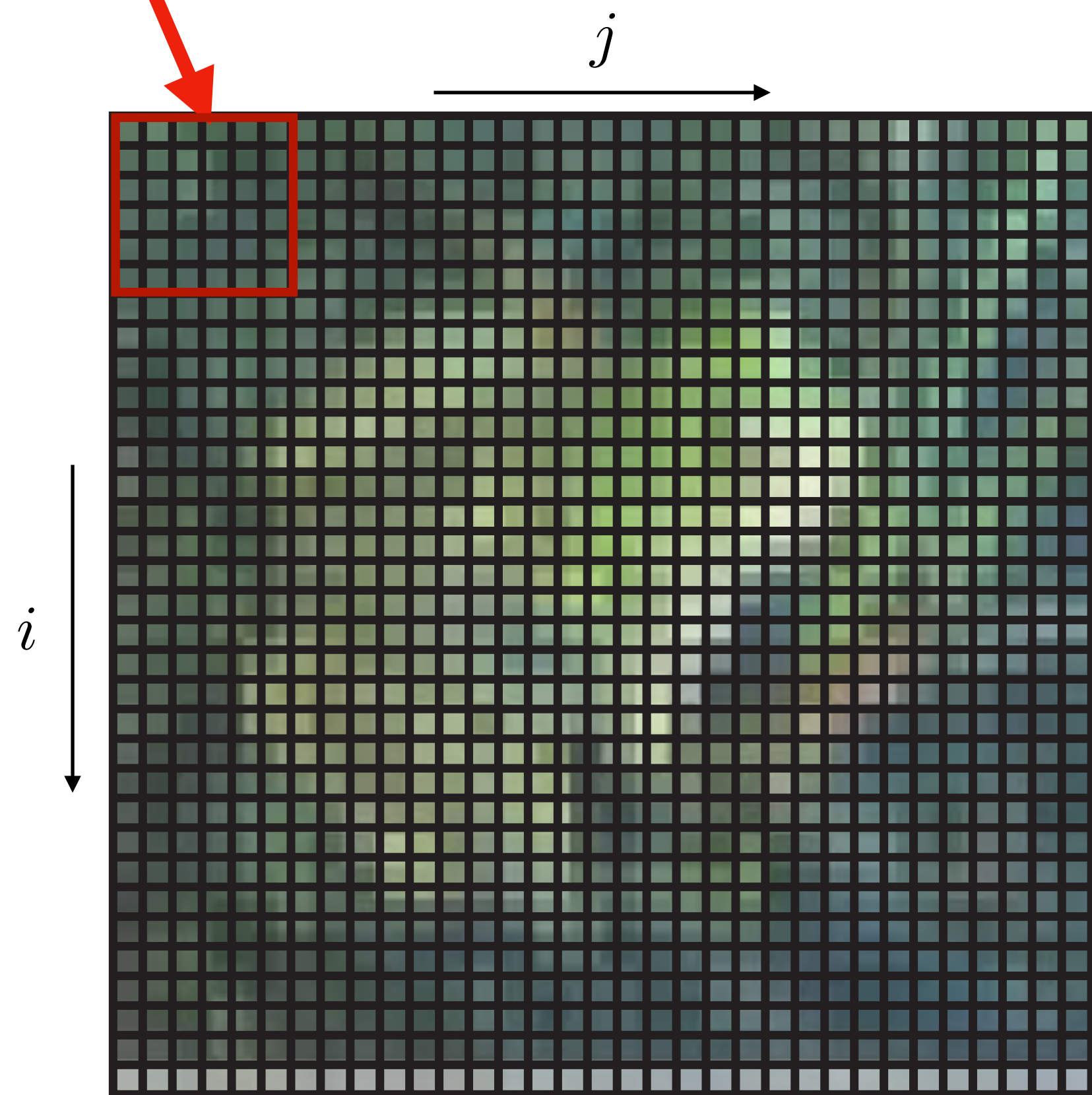
x



z

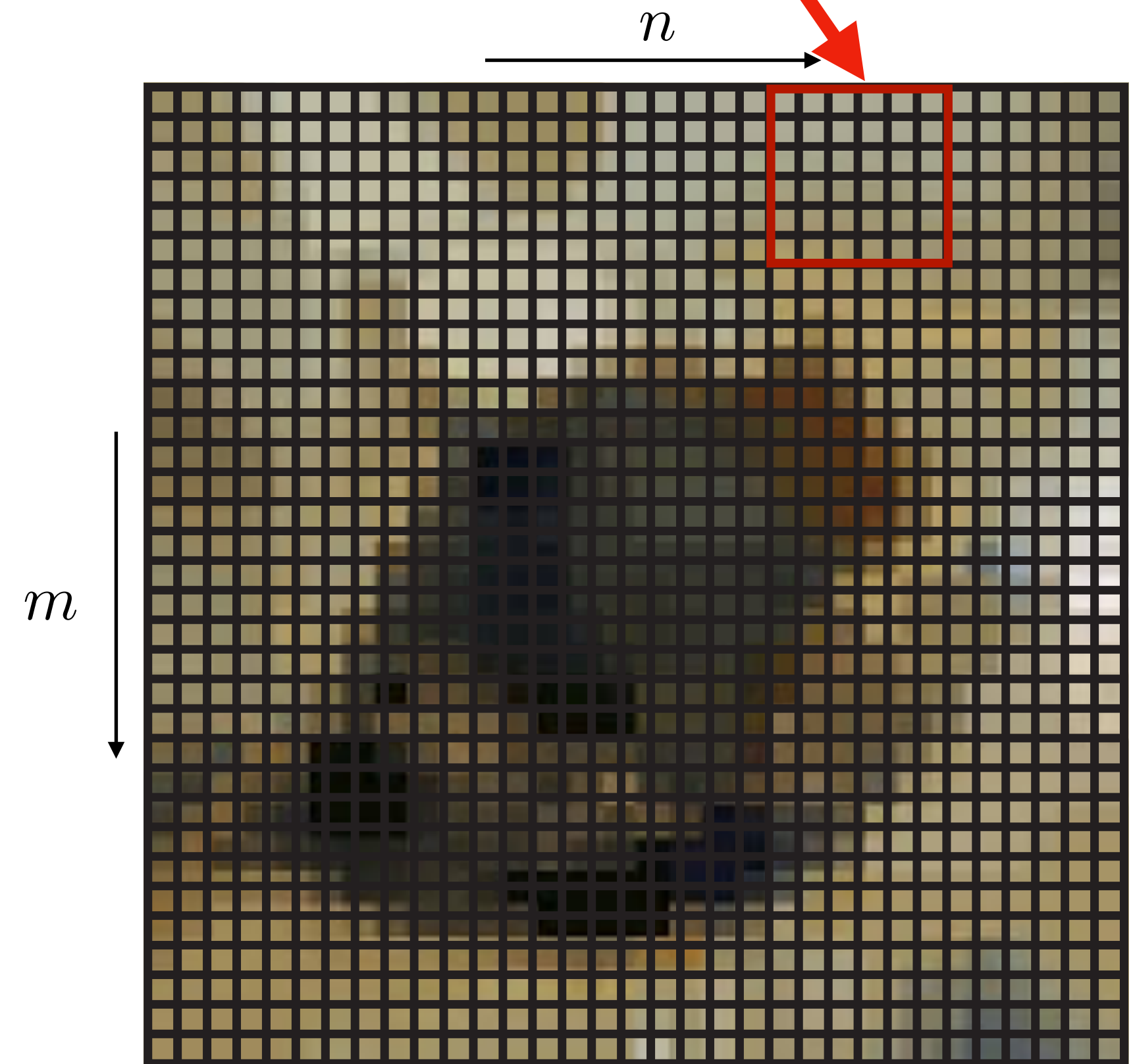
Concatenate Regions

$$c_6(x_{ij}) \in \mathbb{R}^{6 \times 6 \times 3}$$



x

$$c_6(z_{mn}) \in \mathbb{R}^{6 \times 6 \times 3}$$

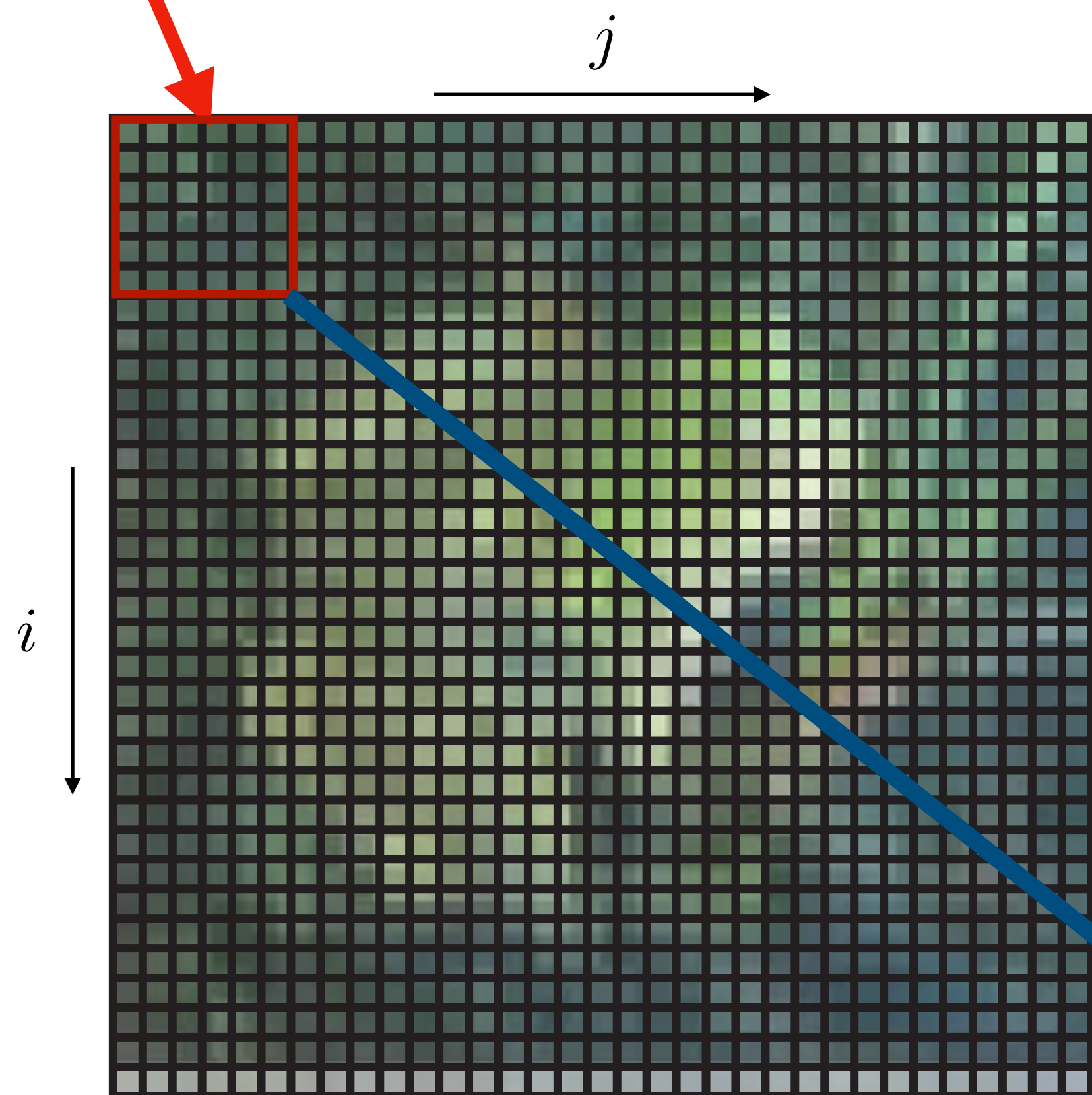


z

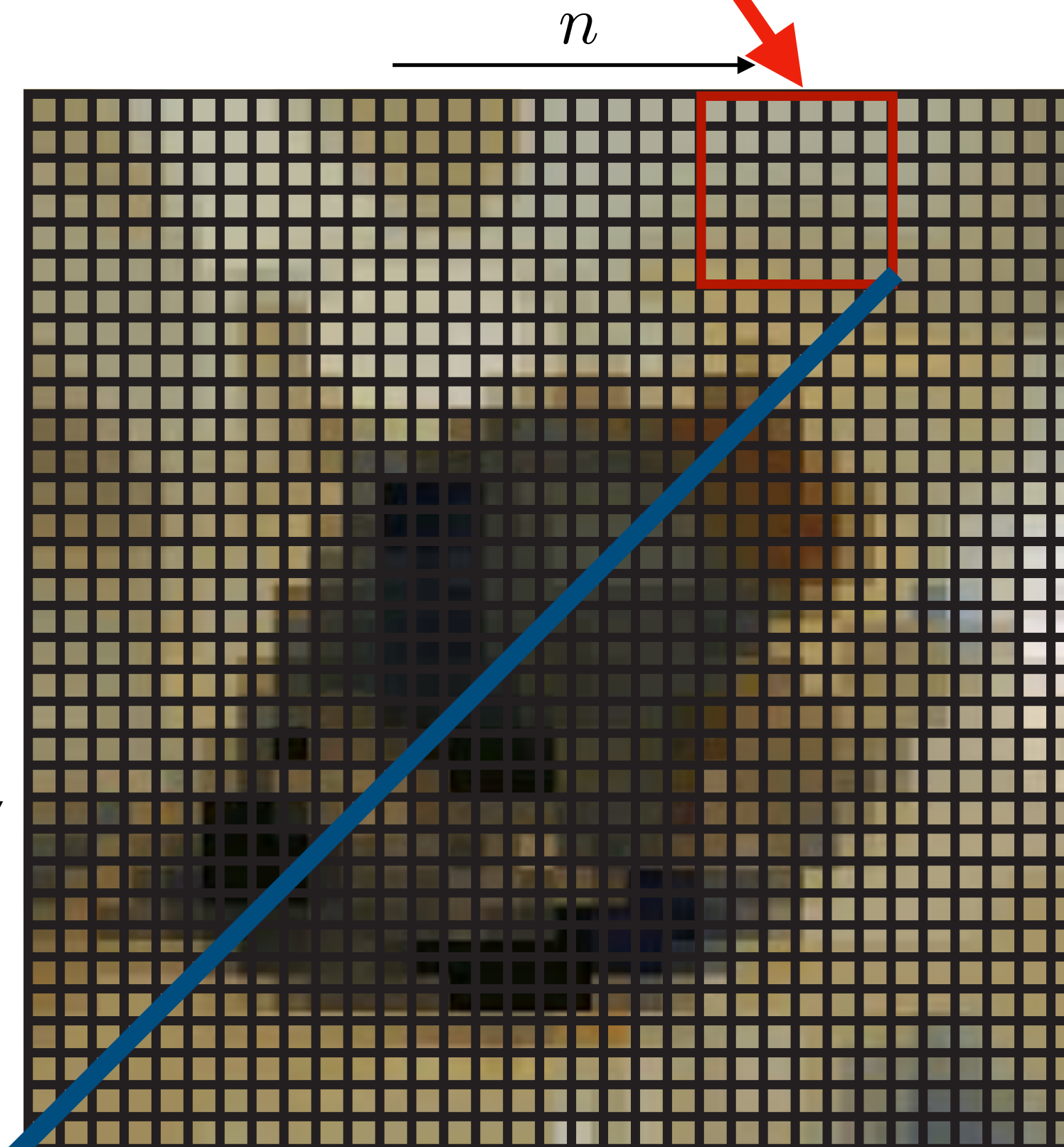
Compare Regions

$$c_6(x_{ij}) \in \mathbb{R}^{6 \times 6 \times 3}$$

$$c_6(z_{mn}) \in \mathbb{R}^{6 \times 6 \times 3}$$



x



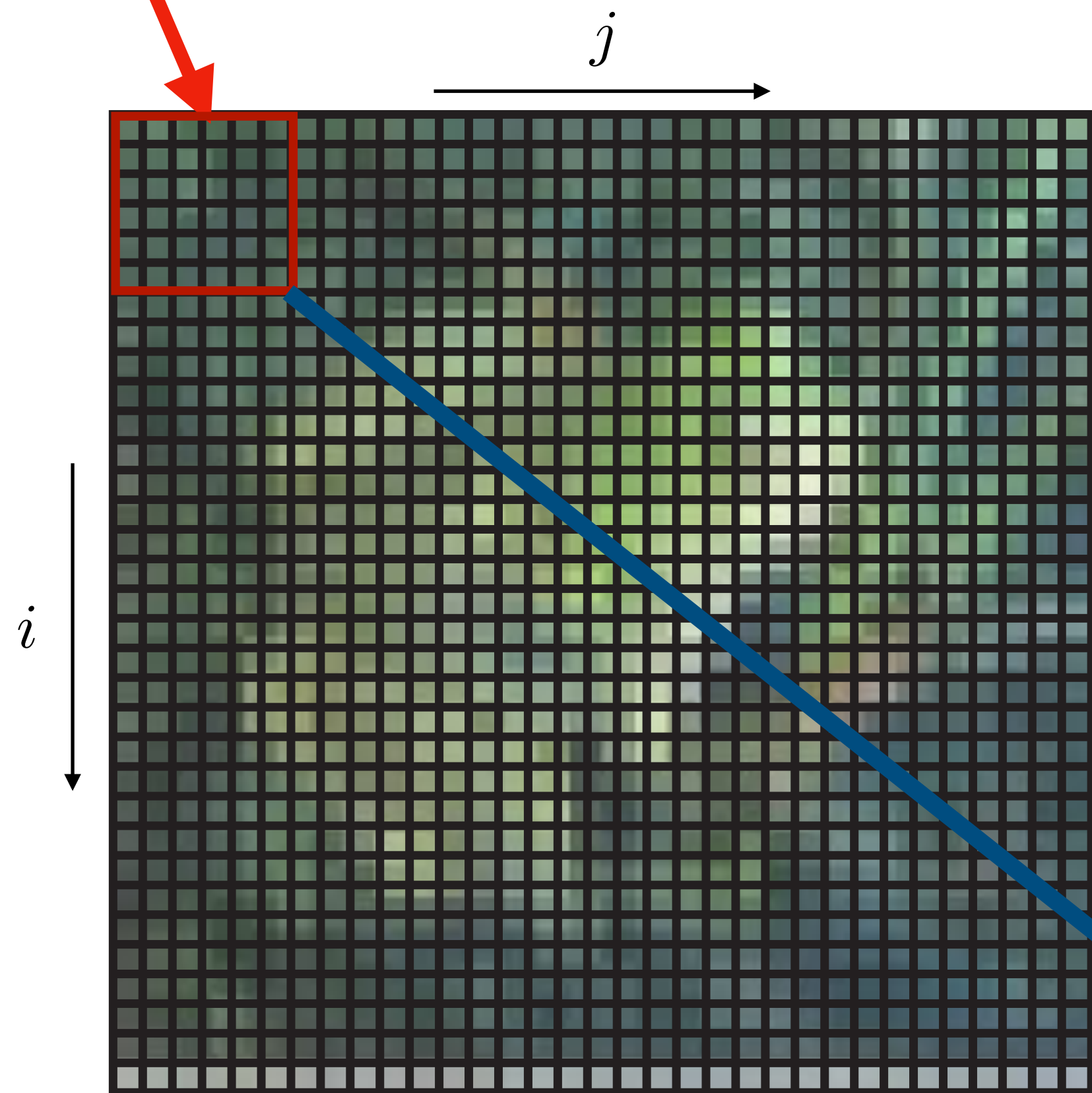
z

$$k(c_6(x_{ij}), c_6(z_{mn}))$$

Compare regions with simple kernel (i.e rbf)

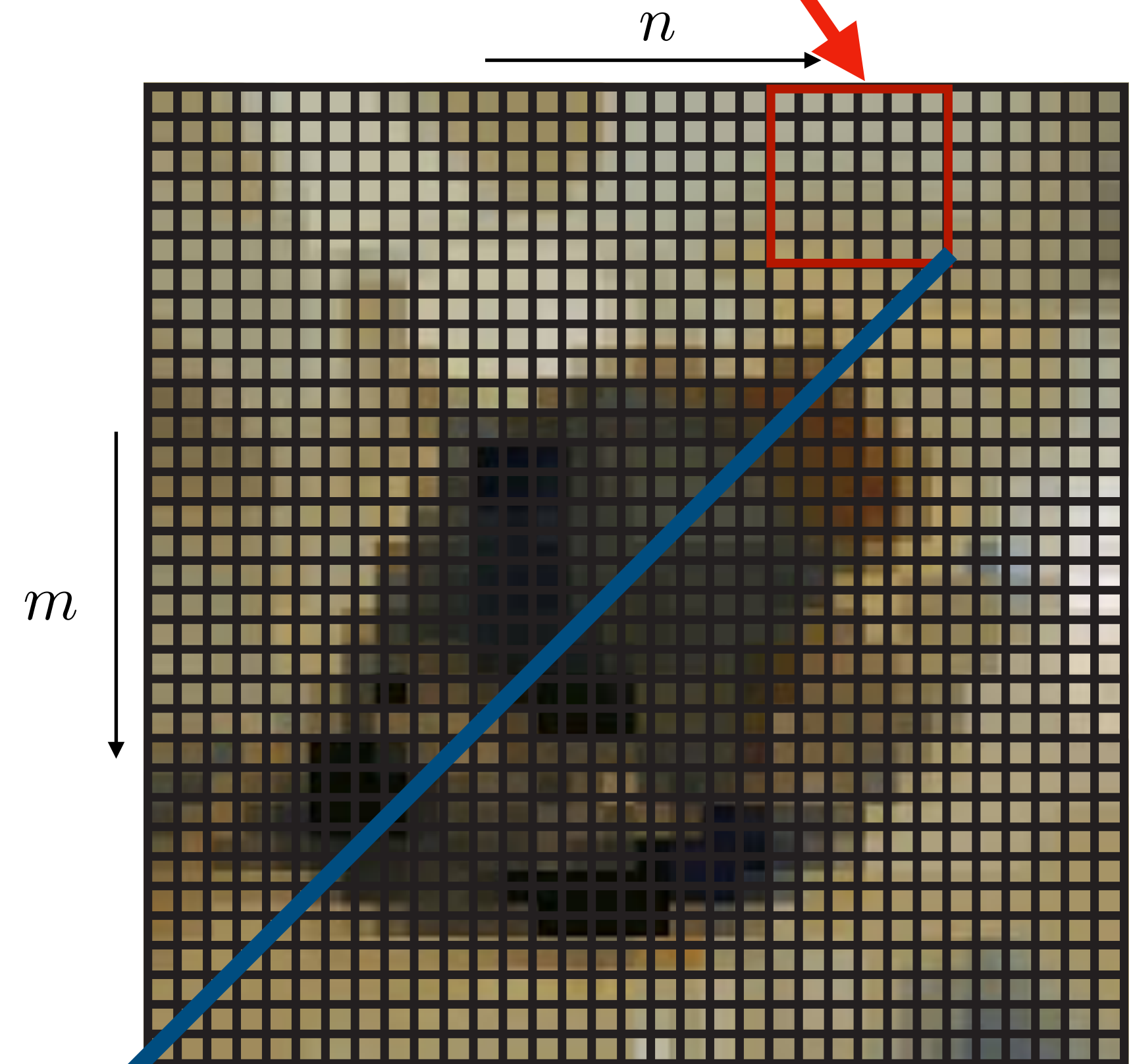
Sum comparisons

$$c_6(x_{ij}) \in \mathbb{R}^{6 \times 6 \times 3}$$



x

$$c_6(z_{mn}) \in \mathbb{R}^{6 \times 6 \times 3}$$



z

$$K(x, y) = \sum_{i,j,m,n} k(c_6(x_{ij}), c_6(z_{mn}))$$

Compare regions with simple kernel (i.e rbf)

3 Steps in Feature Space

- **Concatenate** features
- Apply **nonlinear lift** to concatenated features
- **Sum** lifted features into single feature

Concatenation

$$\left\langle \begin{array}{c} \text{X0} \\ \text{X1} \end{array}, \begin{array}{c} \text{Z0} \\ \text{Z1} \end{array} \right\rangle = \left\langle \begin{array}{c} \text{X0} \\ \text{Z0} \end{array} \right\rangle + \left\langle \begin{array}{c} \text{X1} \\ \text{Z1} \end{array} \right\rangle$$

Nonlinear Lift

$$\left\langle \Phi(\text{x0}), \Phi(\text{z0}) \right\rangle = k(\text{x0}, \text{z0})$$

Let Φ to be feature map of ***k***

Sum

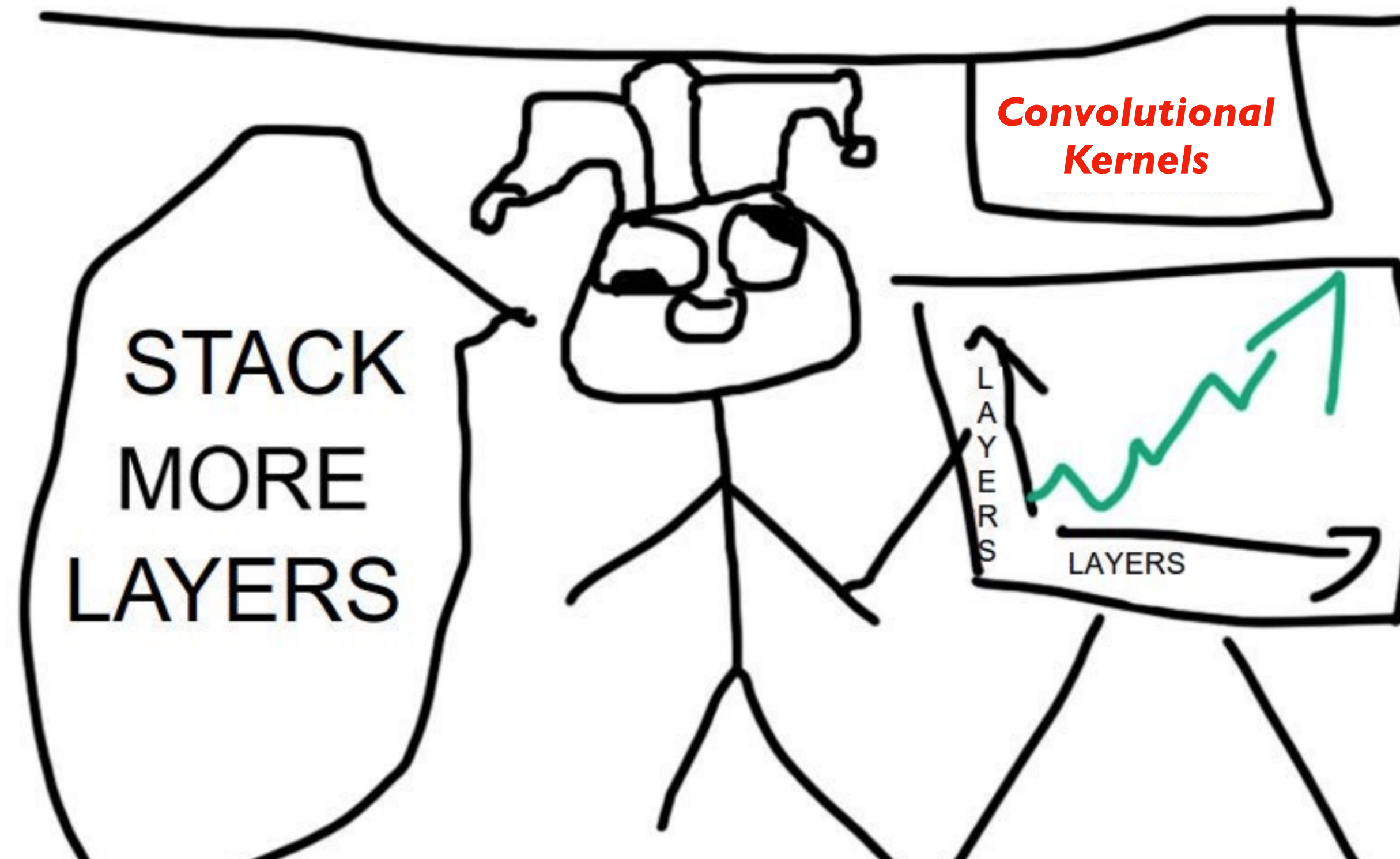
$$\langle x_0 + x_1, z_0 + z_1 \rangle = \langle x_0, z_0 \rangle + \langle x_0, z_1 \rangle + \langle x_1, z_0 \rangle + \langle x_1, z_1 \rangle$$

3 Steps in Feature Space

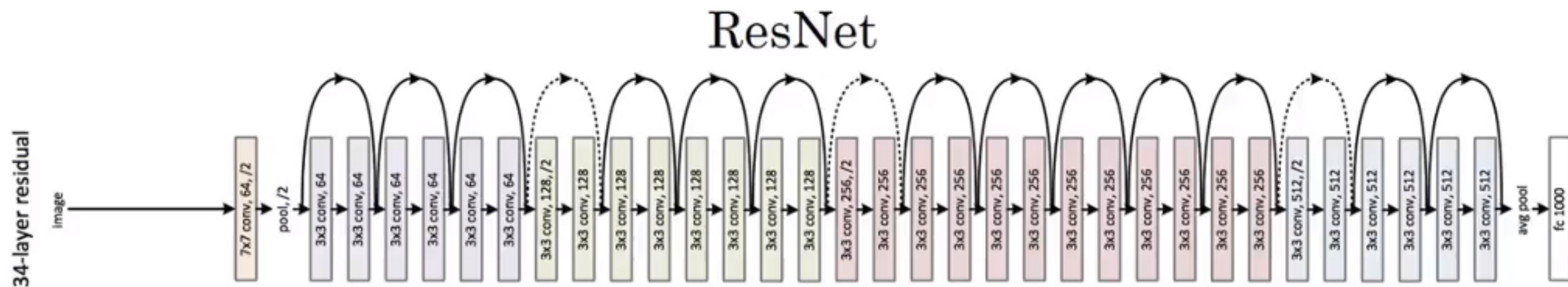
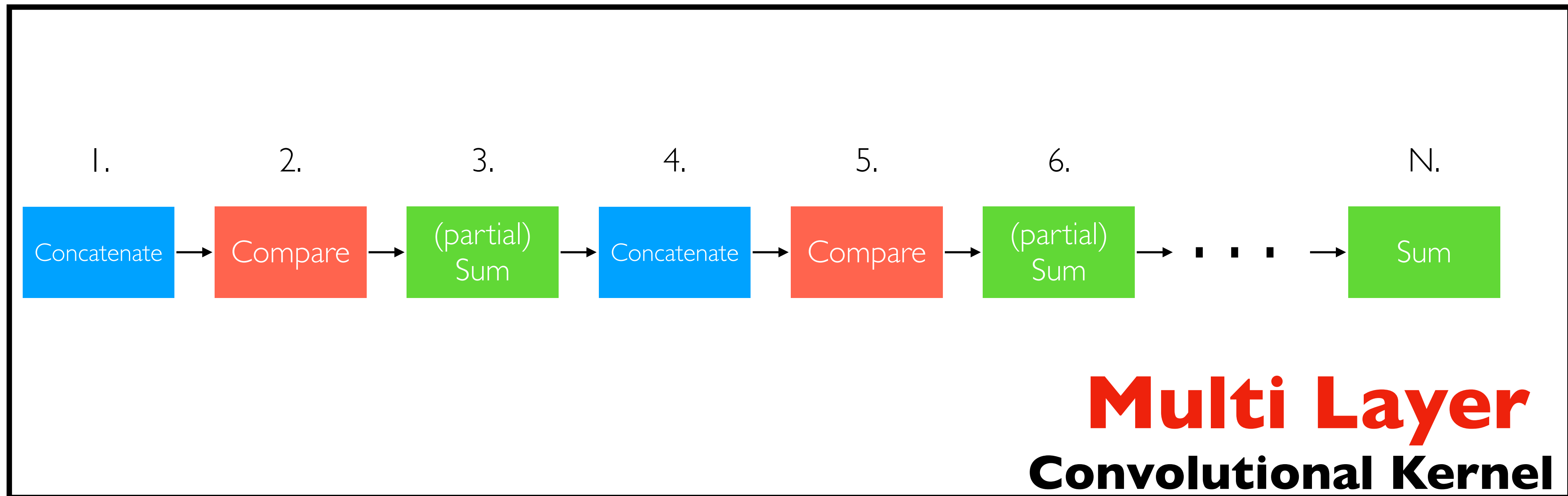
- **Concatenate** features *Convolution*
- Apply **nonlinear lift** to concatenated features *Rectify*
- **Sum** lifted features into single feature *Pool*

Convolutional Kernel

How to iterate the kernel?

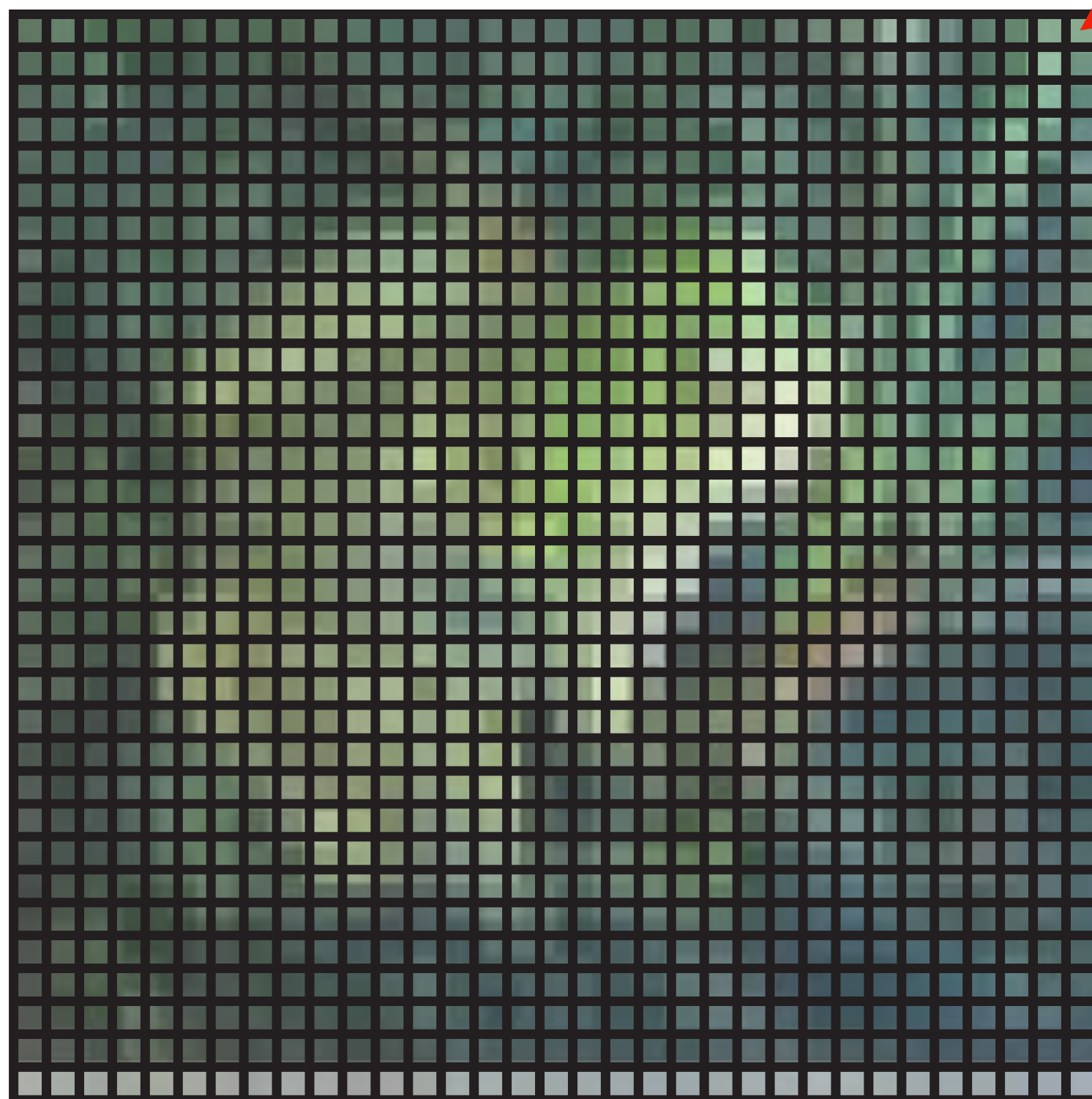


Multilayer Kernel



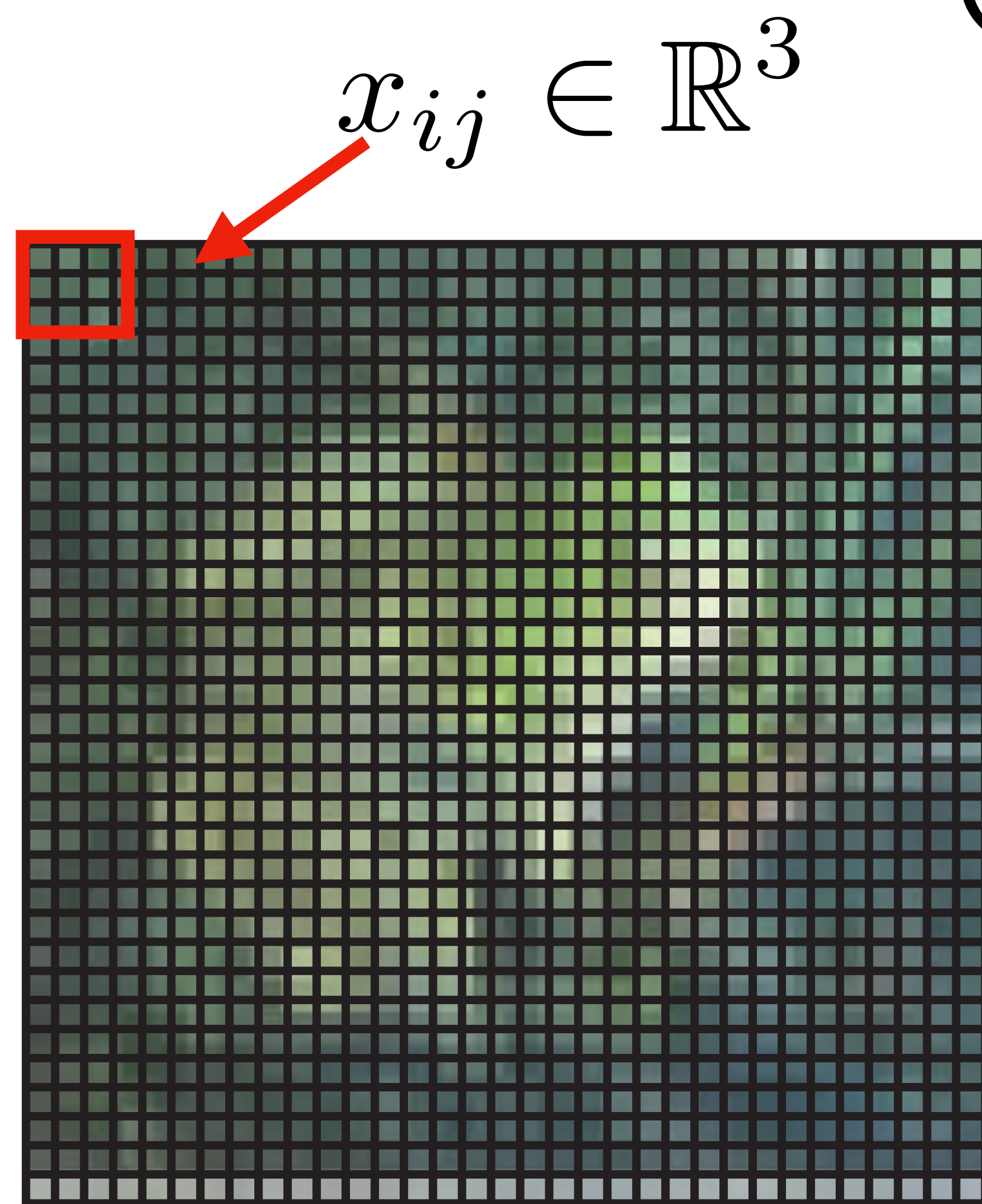
Constructing kernel from
bag of vectors

Box of vectors. $x_{ij} \in \mathbb{R}^3$



$\mathcal{X}$

Concatenation

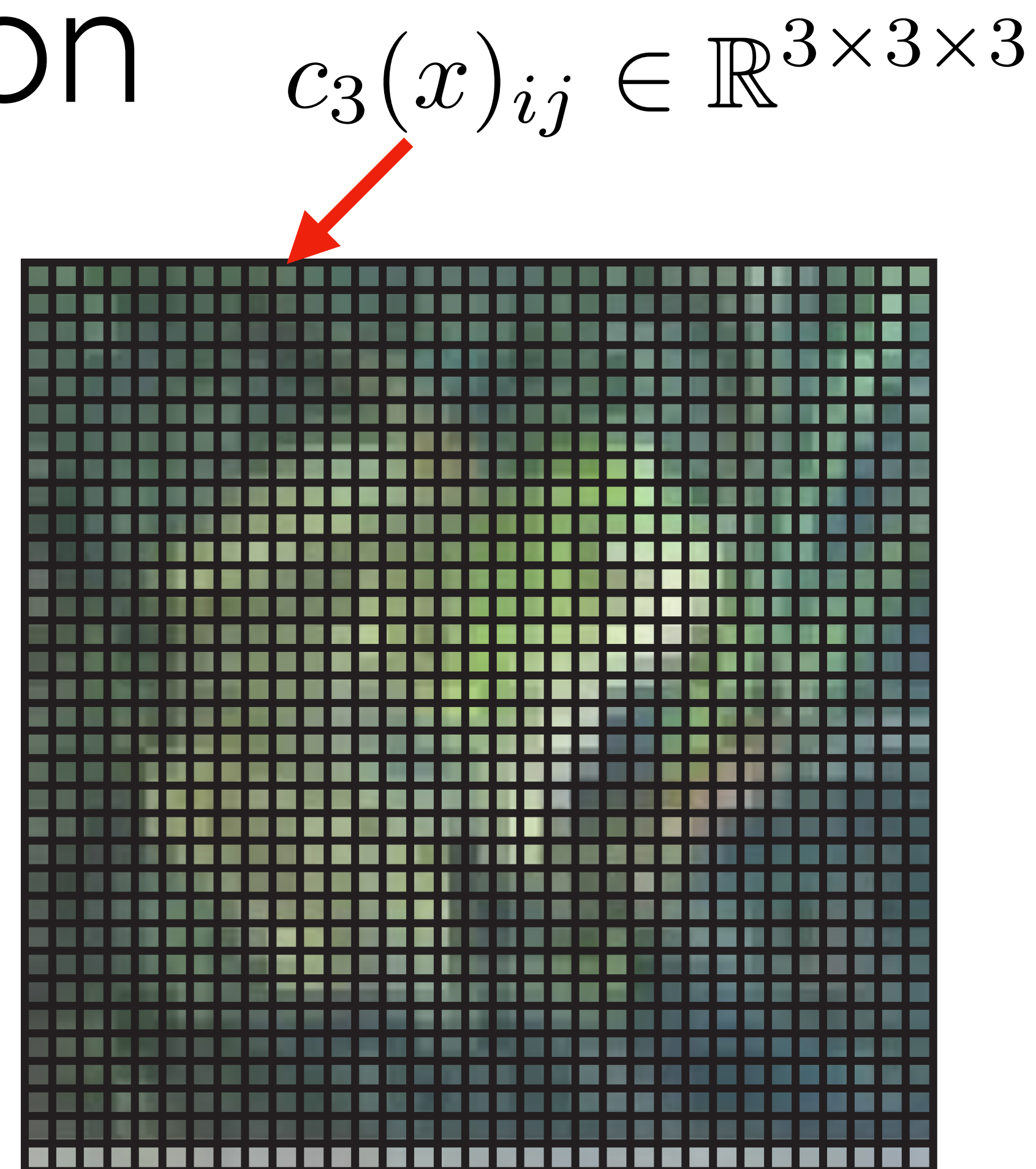


$$x_{ij} \in \mathbb{R}^3$$

x



c_3



$$c_3(x)_{ij} \in \mathbb{R}^{3 \times 3 \times 3}$$

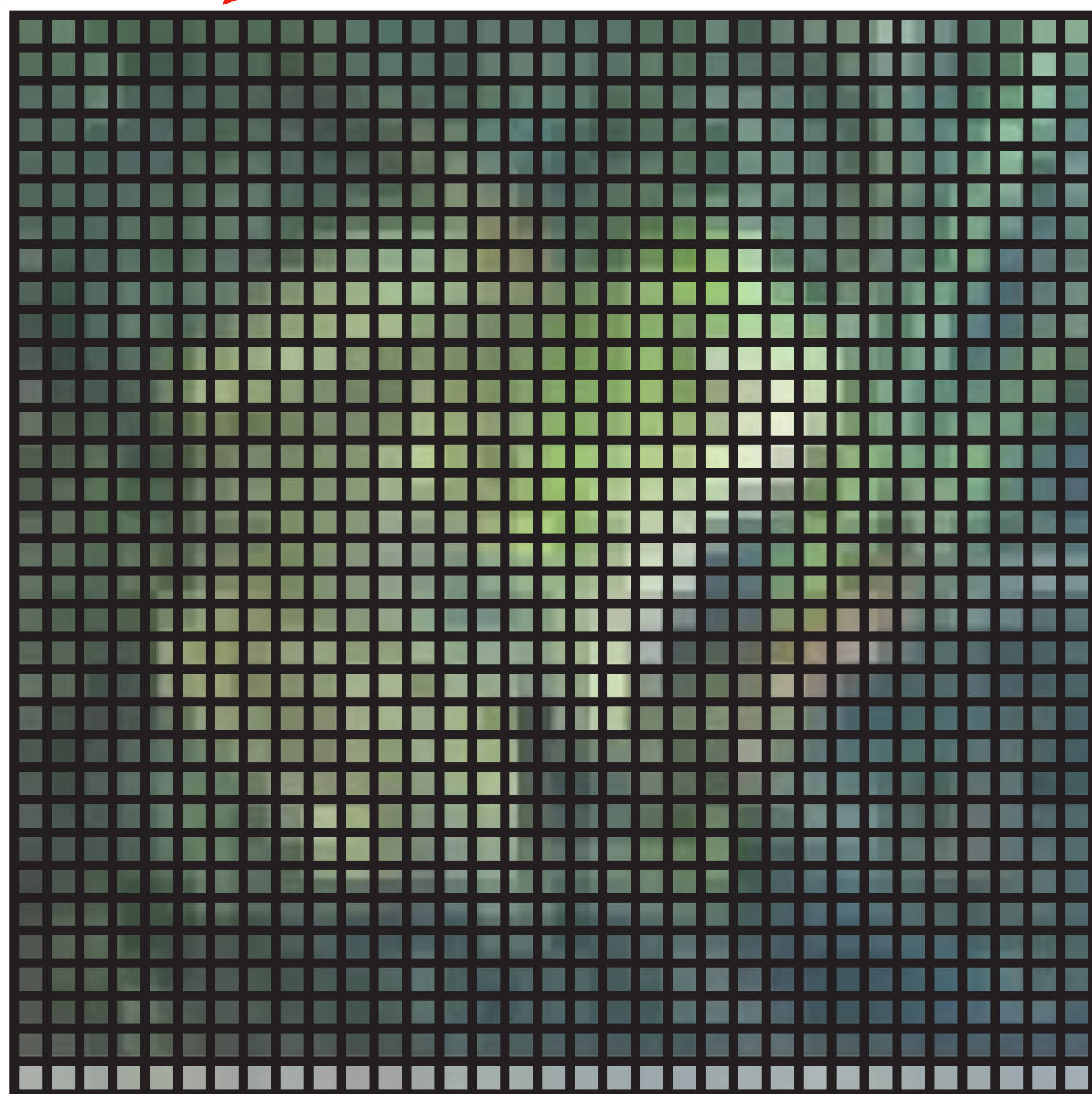
$c_3(x)$

$$c_3(x)_{ij} \in \mathbb{R}^{3 \times 3 \times 3}$$

Embedding

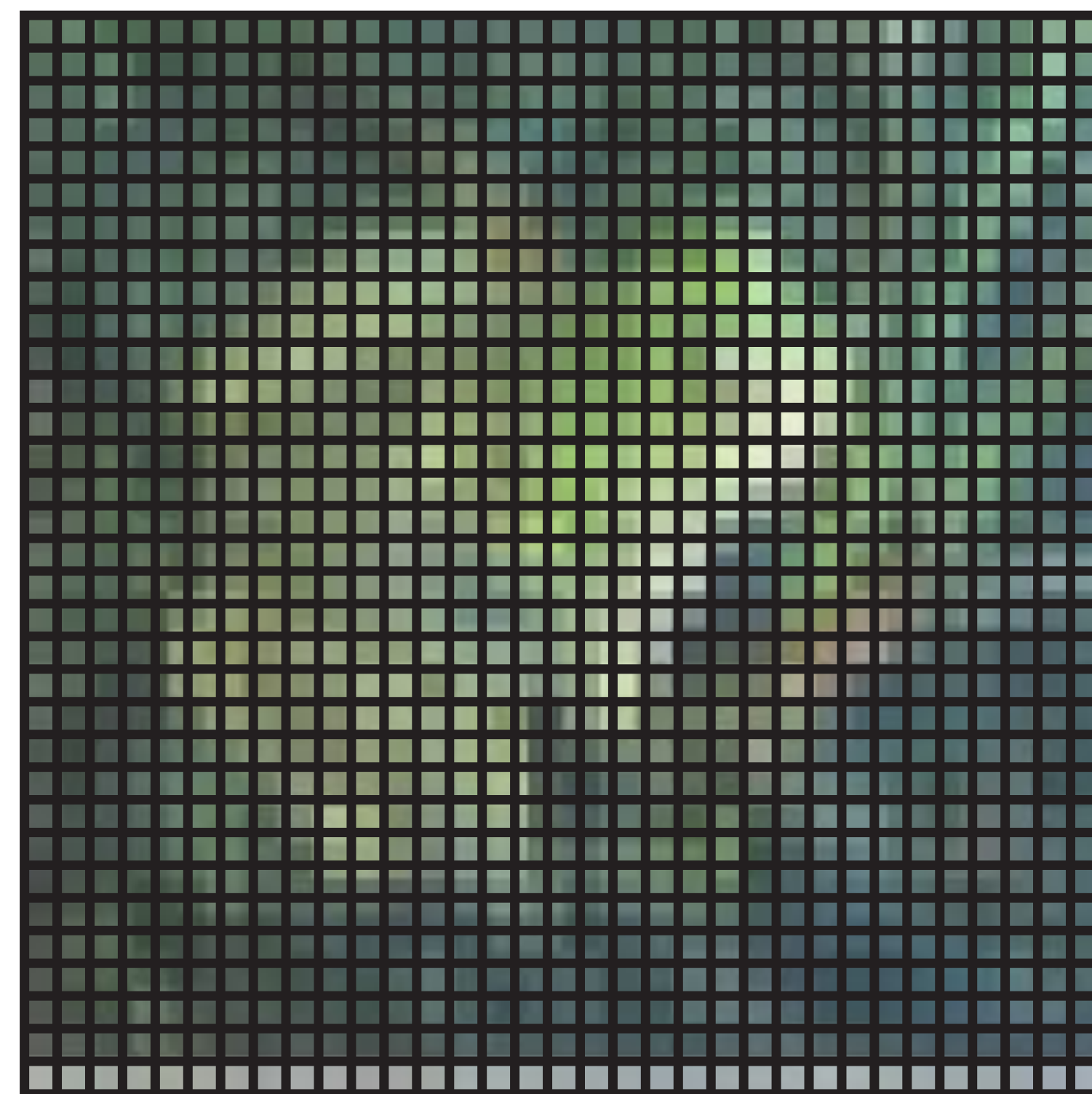
Apply nonlinear “lift” to every vector

$$\varphi(c_3(x)_{ij})$$



$$c_3(x)$$

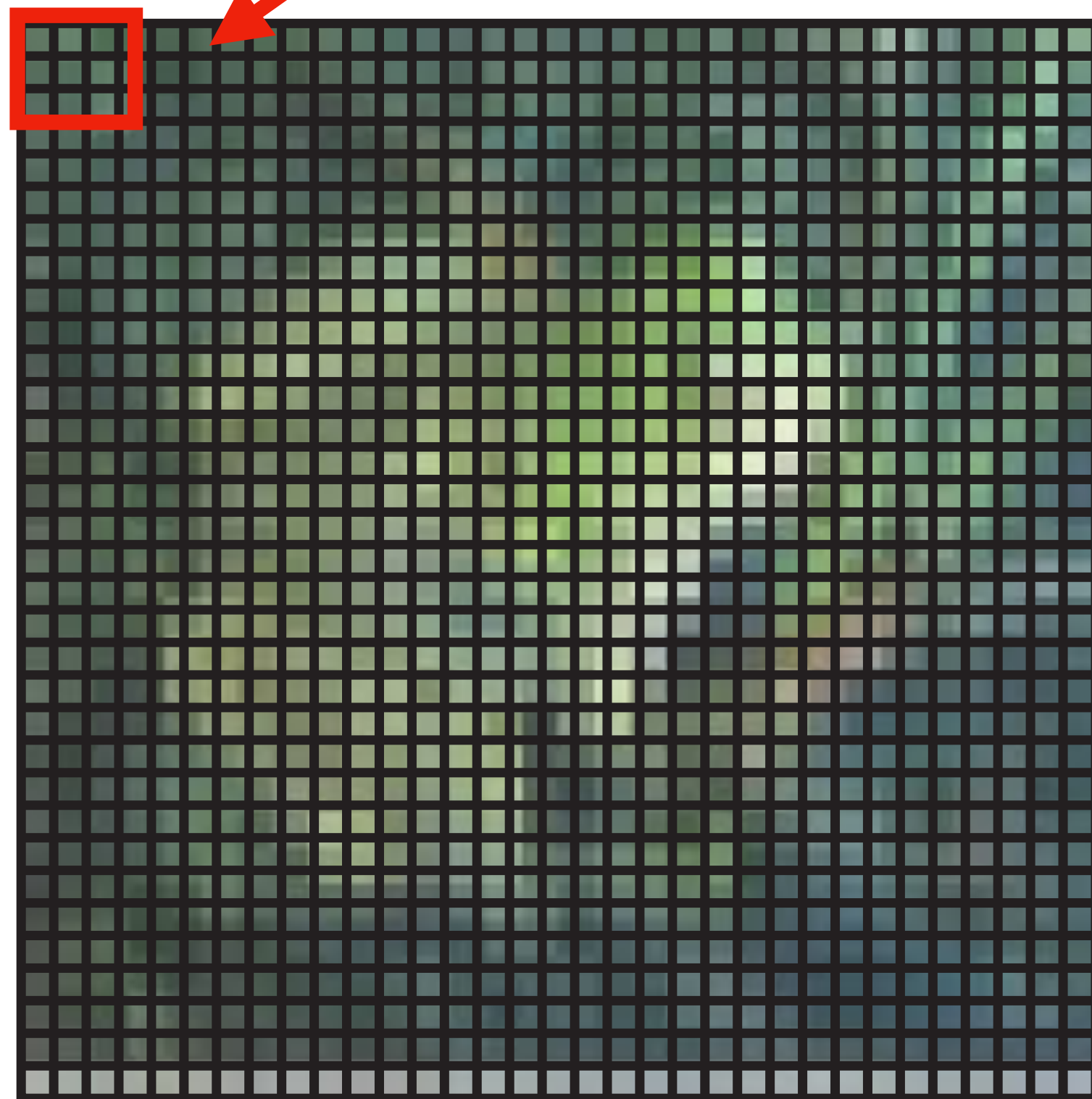
Φ



$$\Phi(c_3(x))$$

Concatenation...again

$$\varphi(c_3(x)_{ij})$$

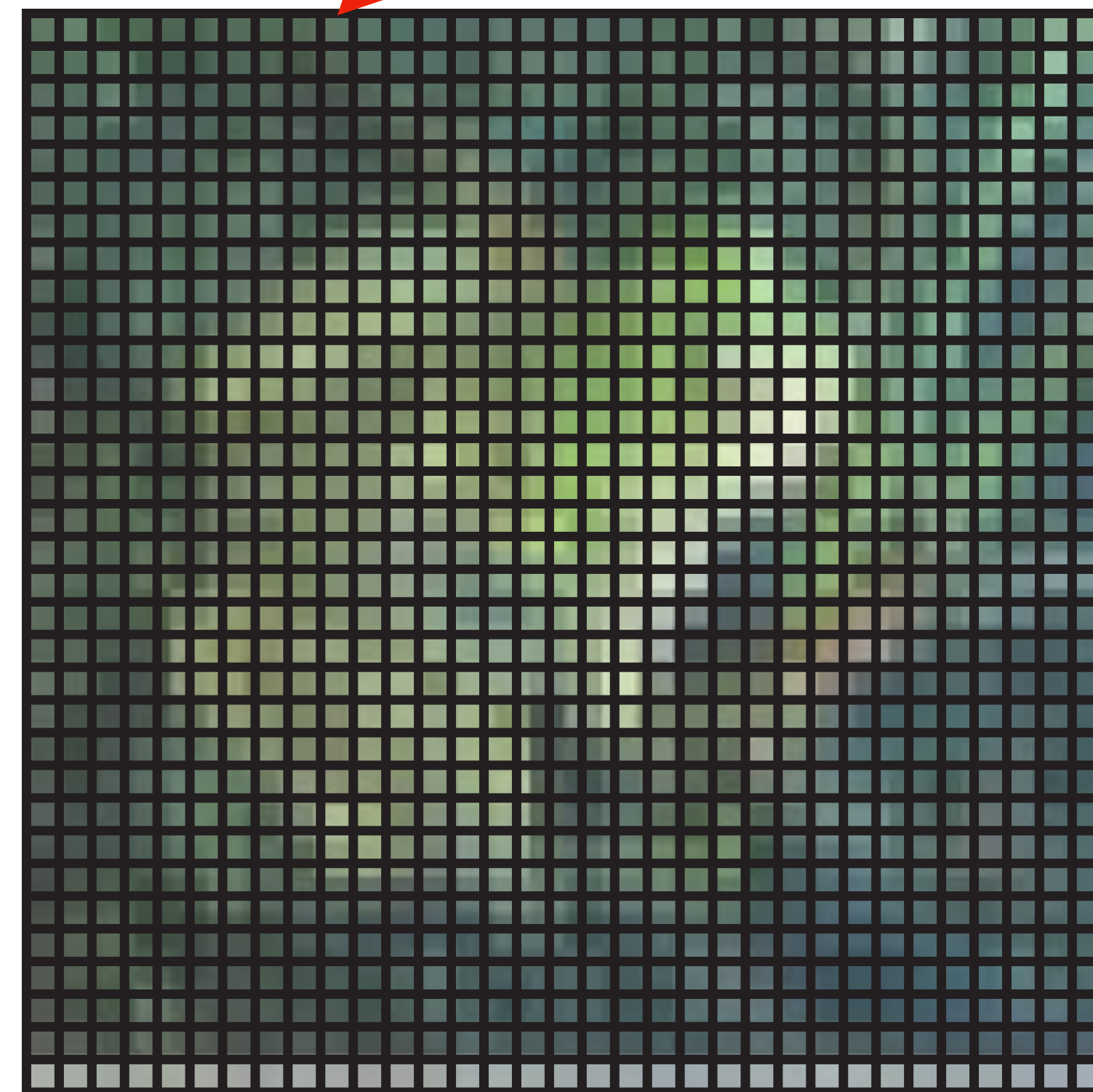


$$\Phi(c_3(x))$$

$$c_3$$

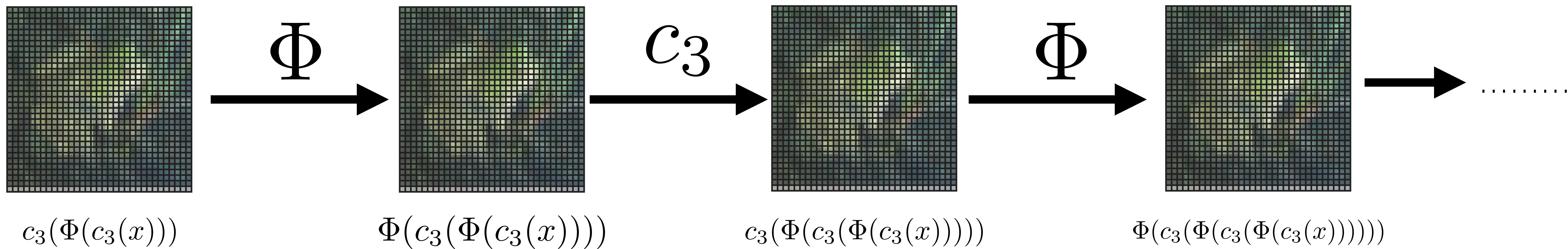


$$c_3(\varphi(c_3(x)))_{ij}$$



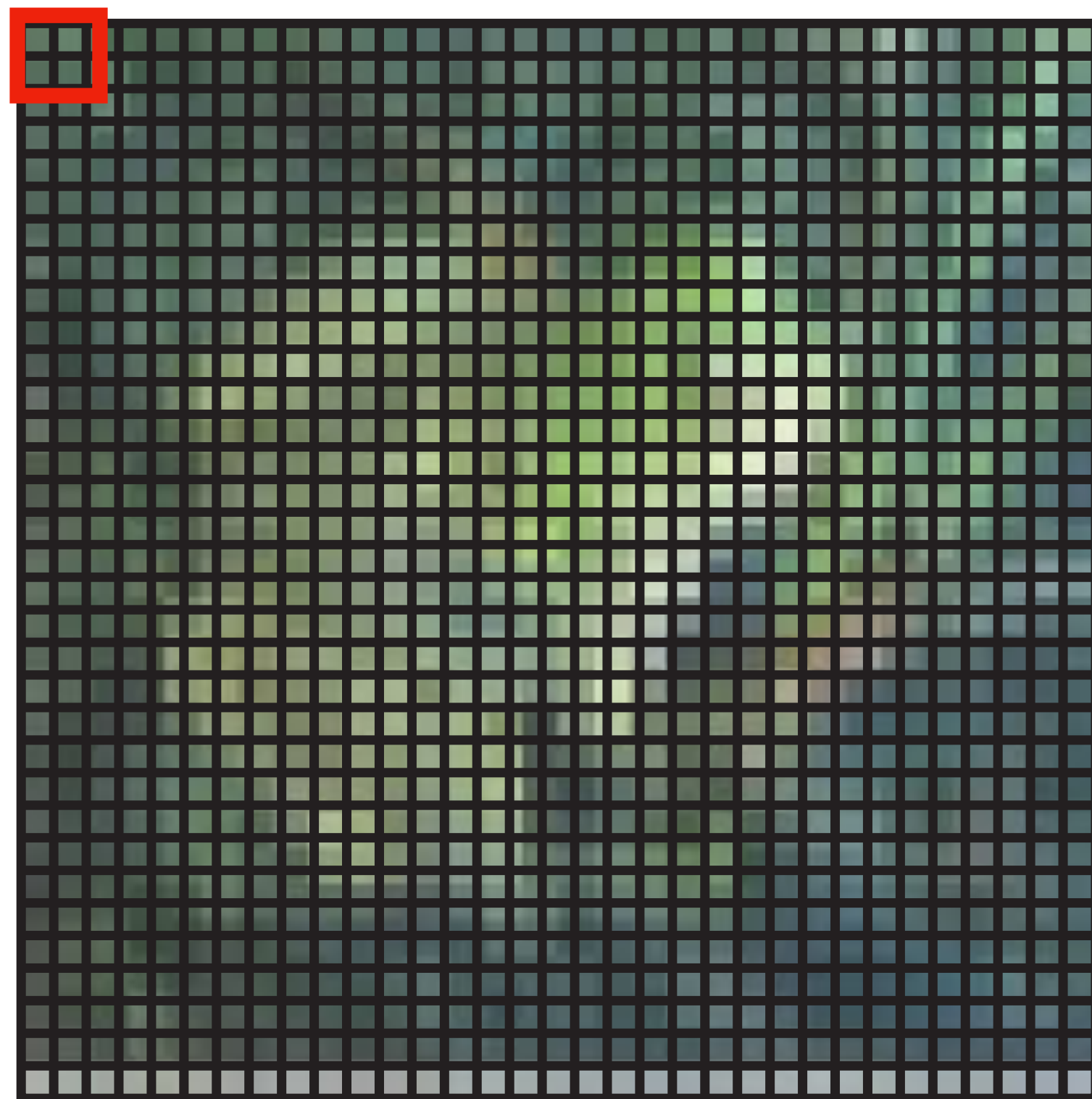
$$c_3(\Phi(c_3(x)))$$

And repeat...

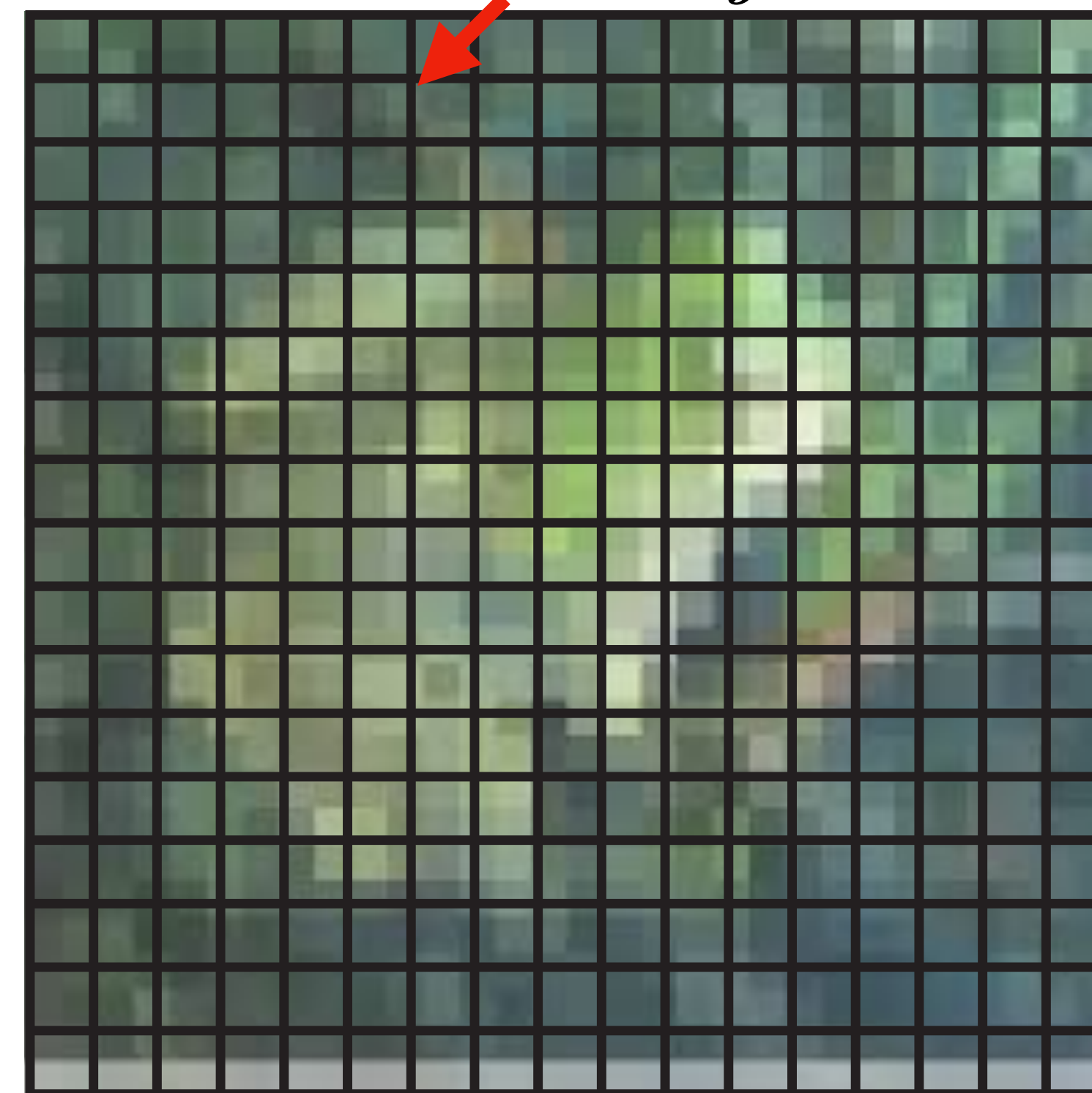
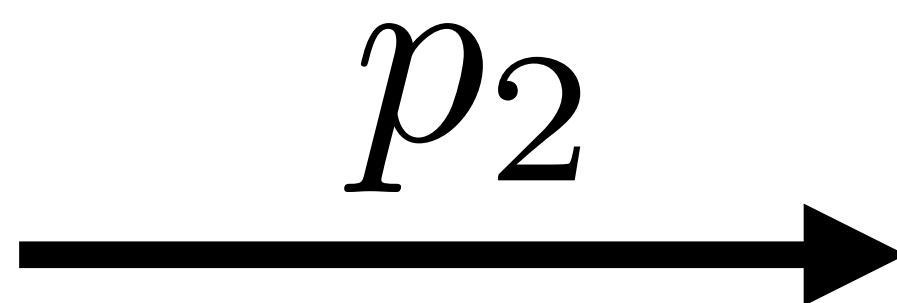


Downsample

$$p_2(\Phi^{(n)}(x))_{ij} = \sum_{w=2i}^{2i+2} \sum_{z=2j}^{2j+2} \Phi^{(n)}(x)_{wz}$$

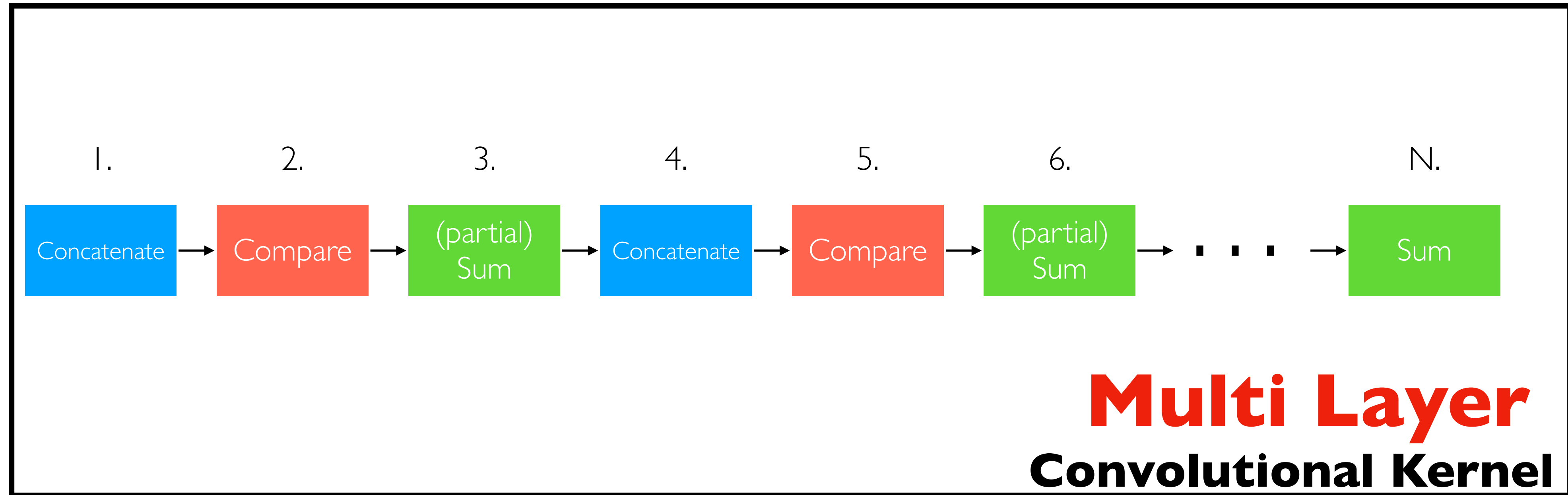


$\Phi^{(n)}(x)$



$p_2(\Phi^{(n)}(x))$

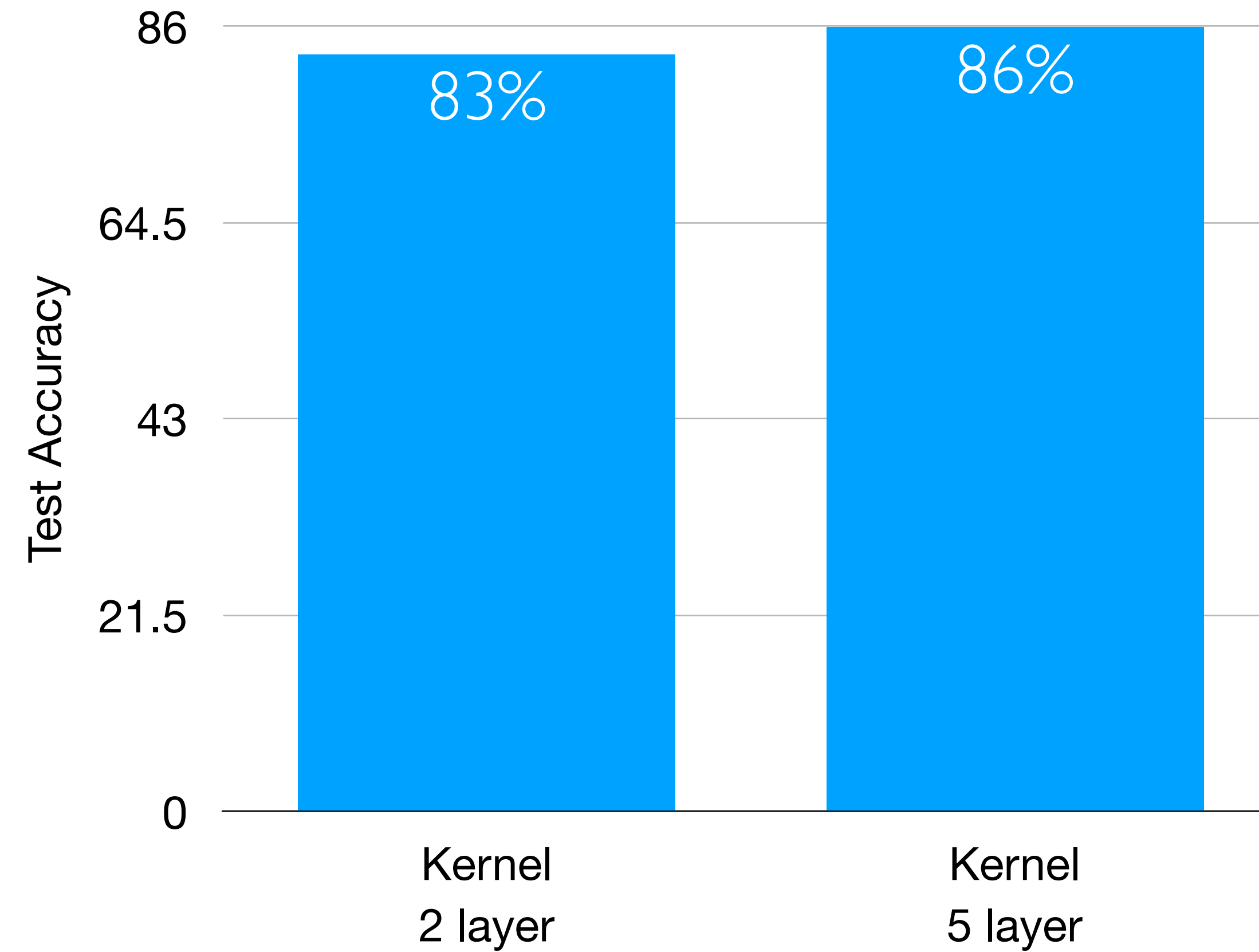
Iterate!



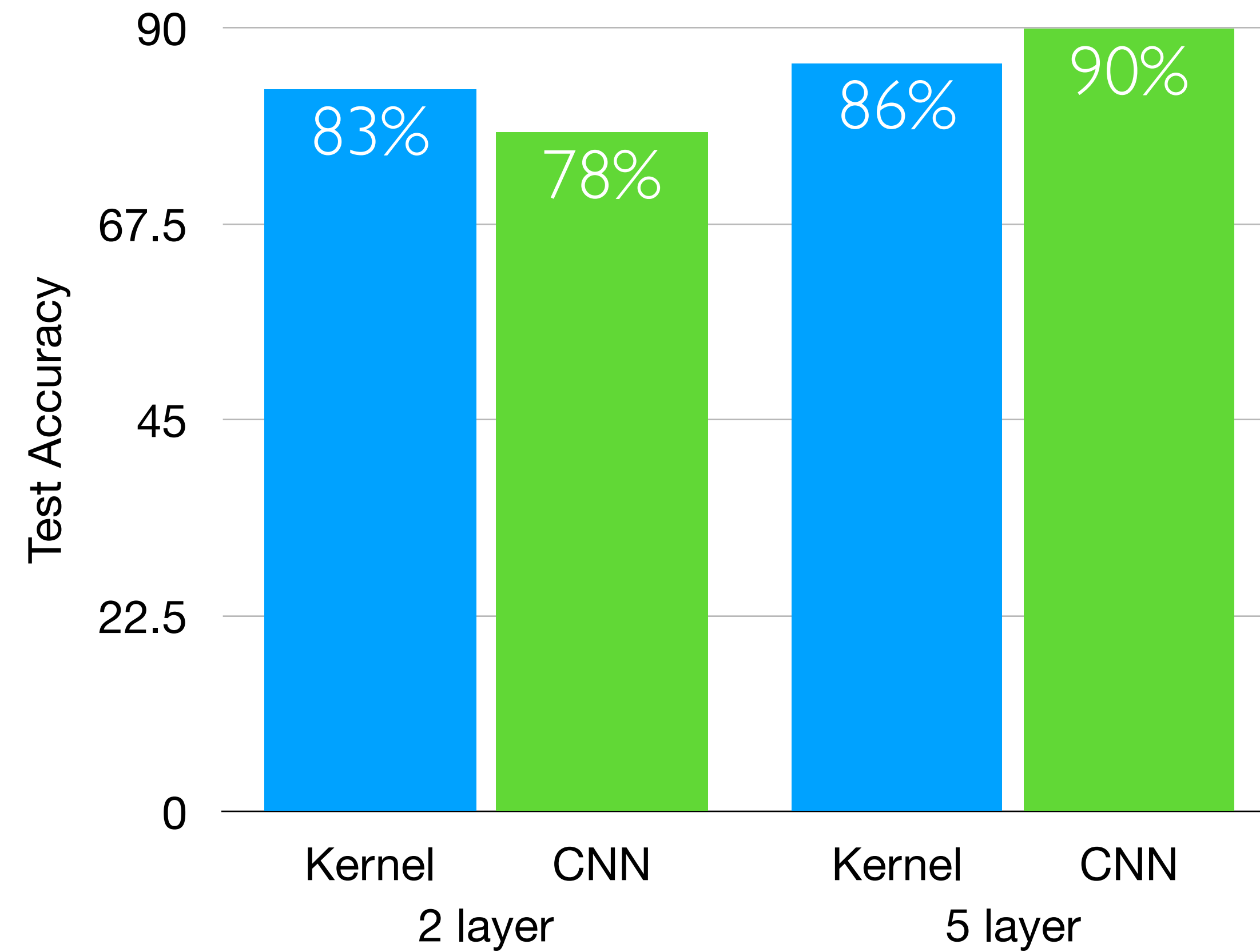
Repeat until spatial dimension is 1

Does it work?

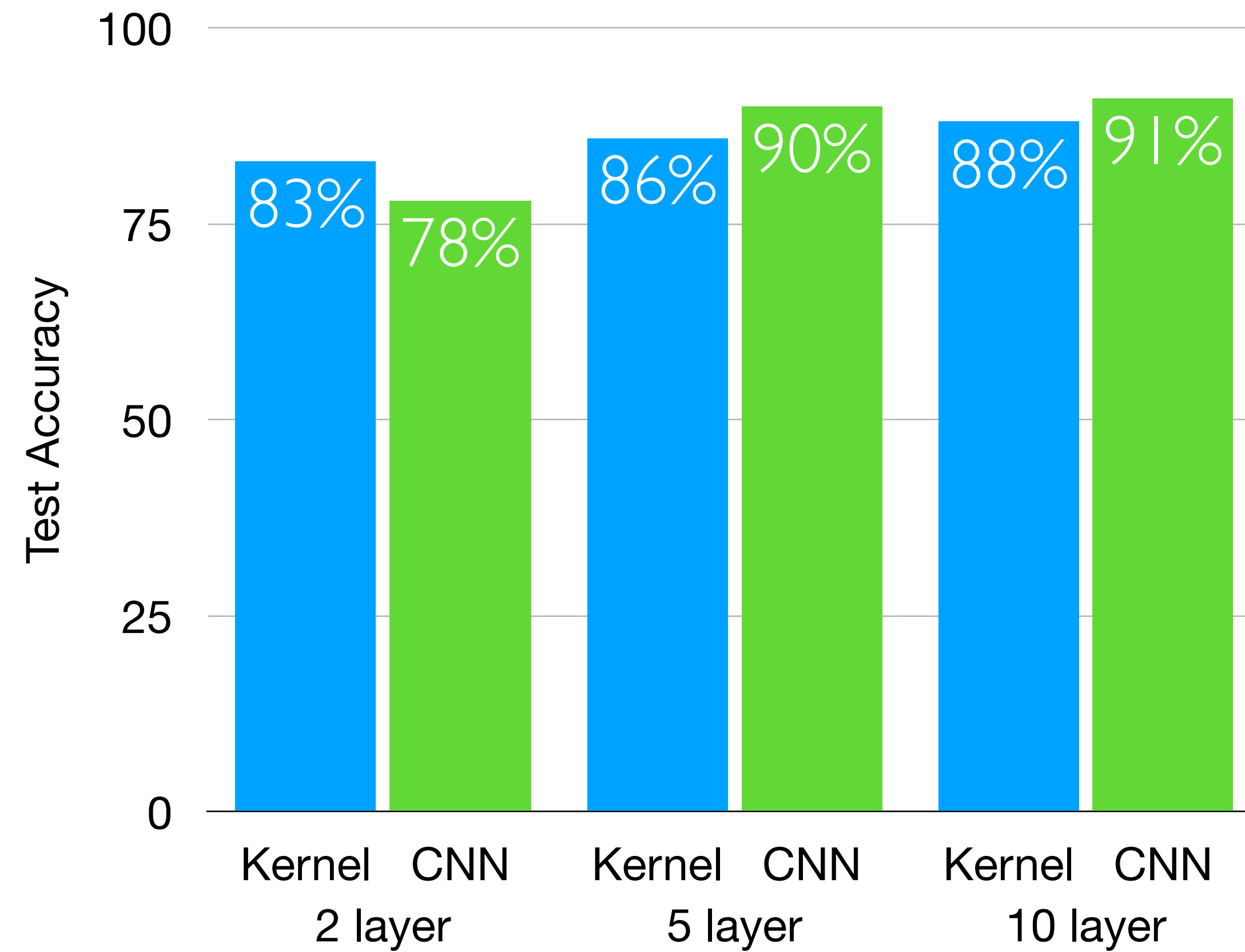
Cifar-10



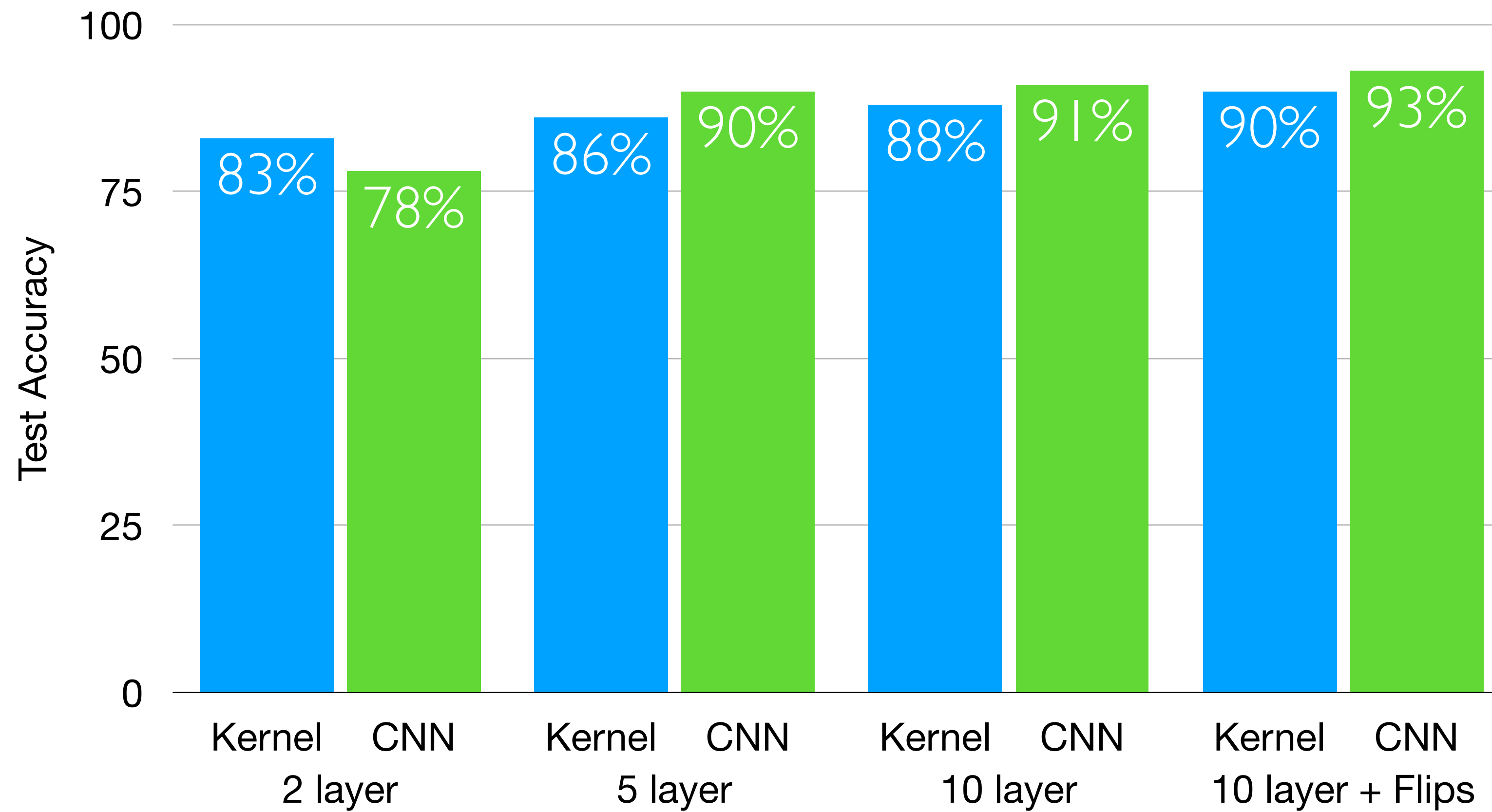
Cifar-10



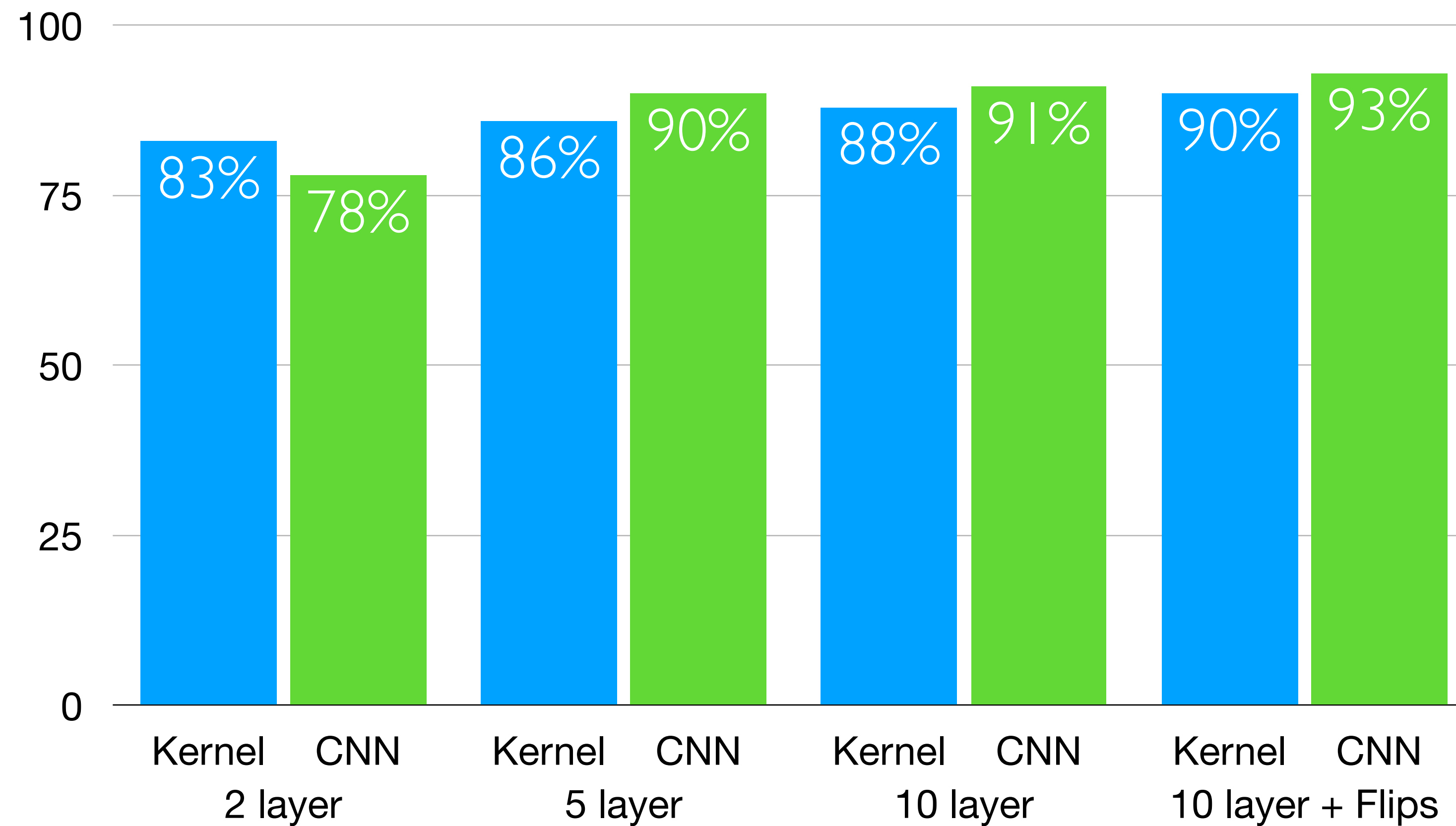
Cifar-10



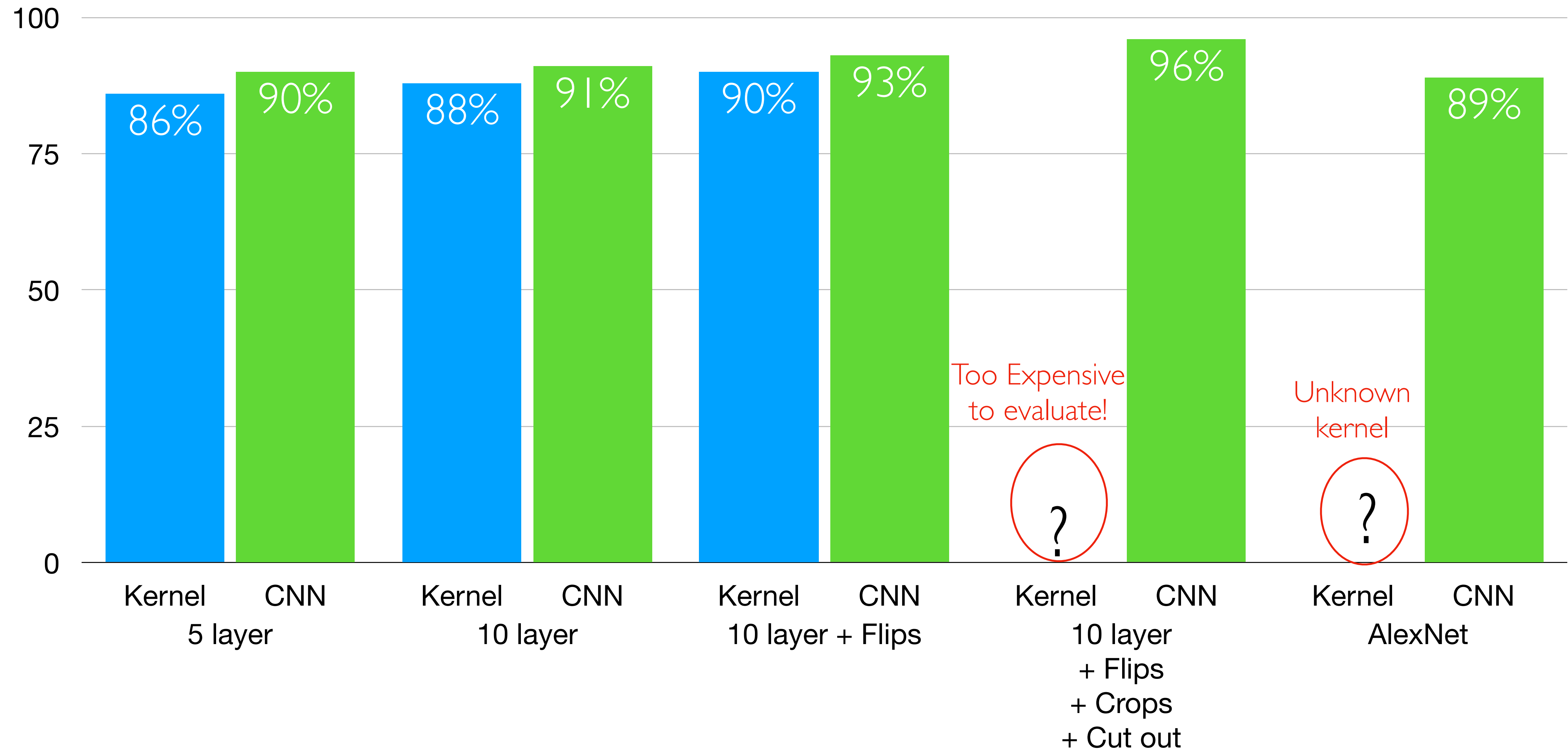
Cifar-10



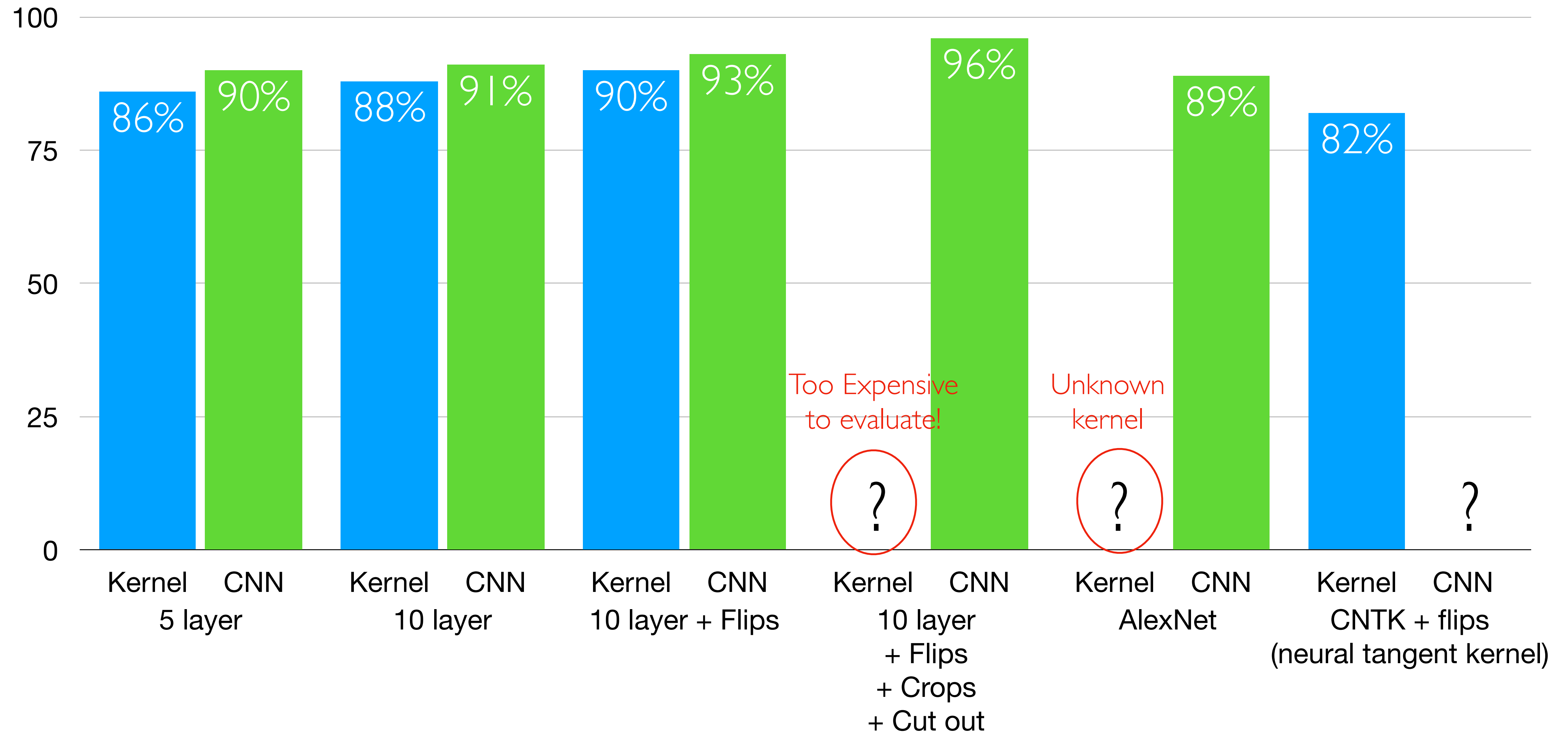
Cifar-10



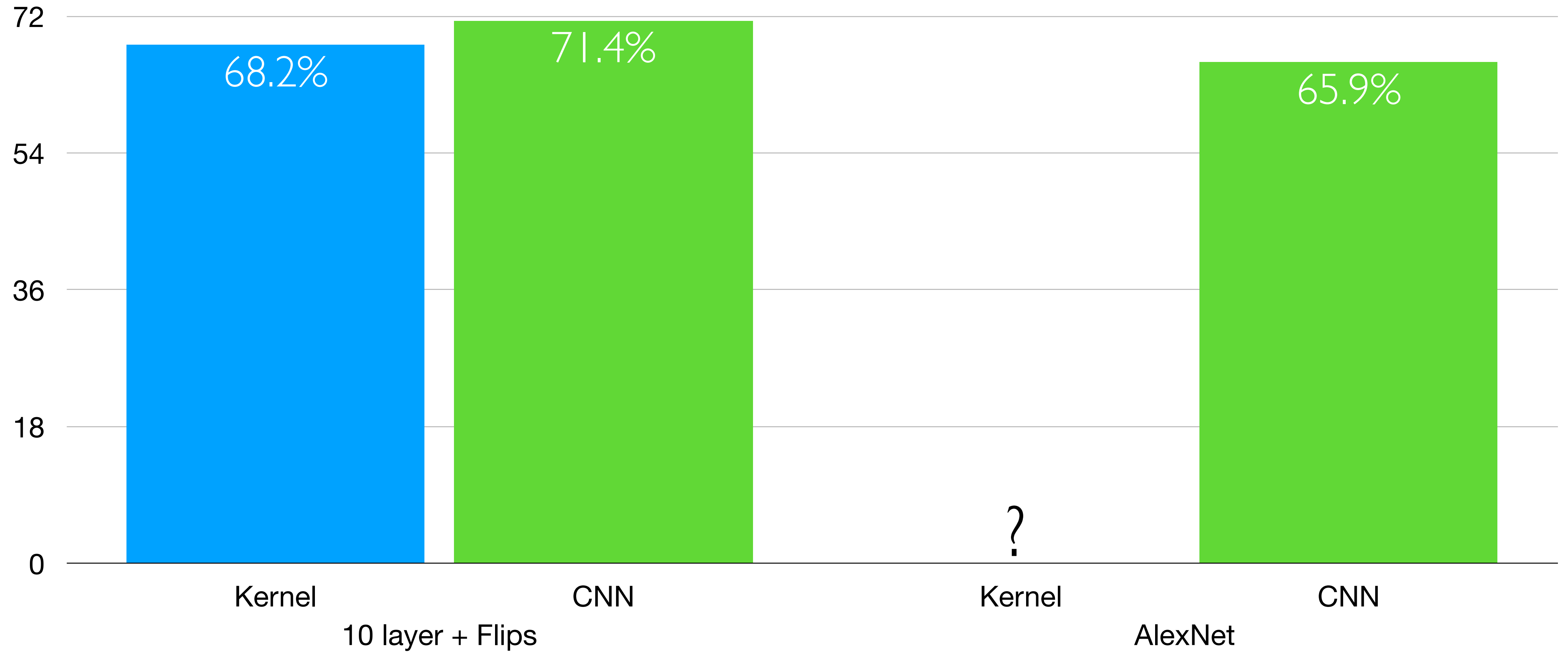
Cifar-10



Cifar-10

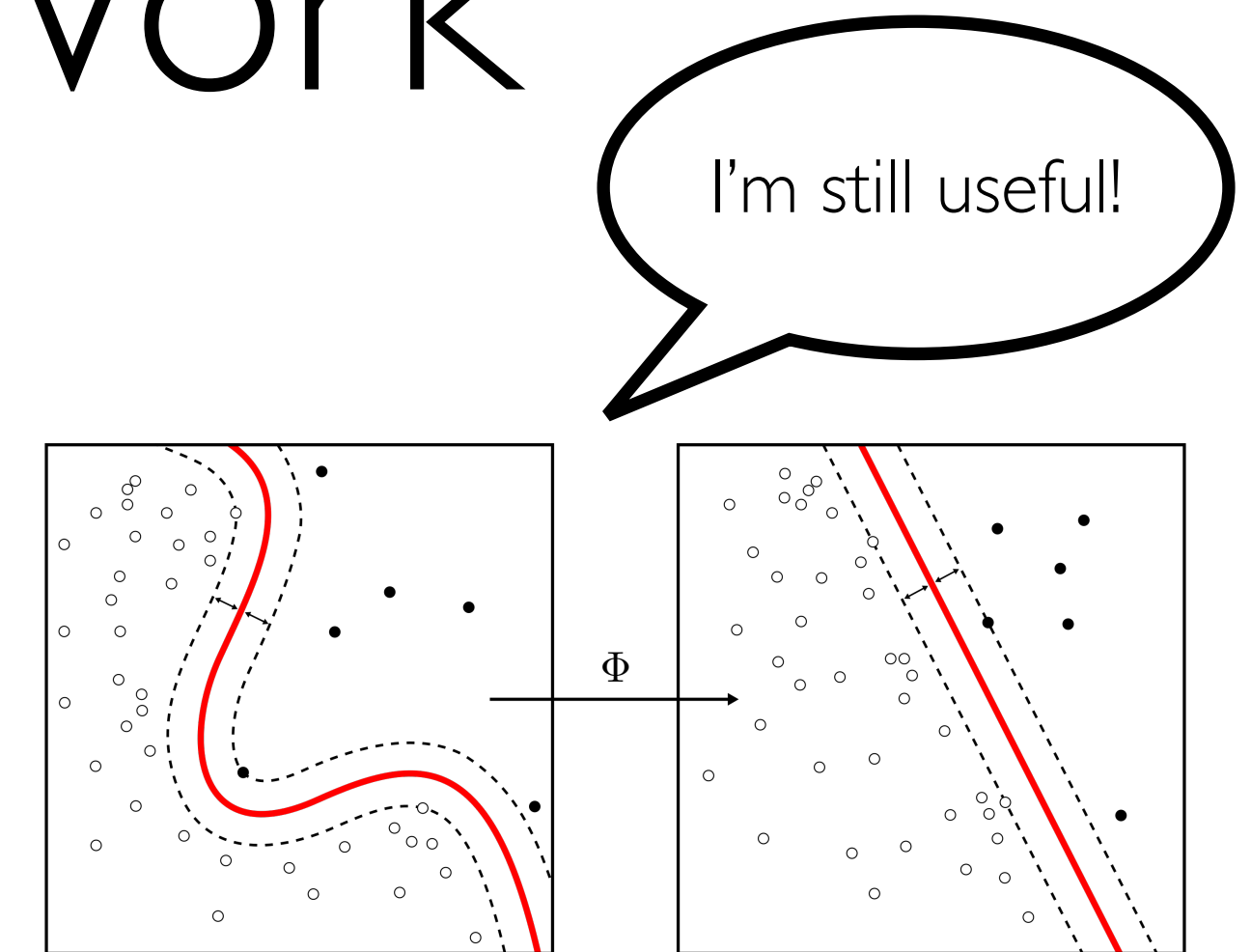


Cifar-100



Conclusions & Future Work

- Kernel methods can be improved by incorporating **convolutional structure**
- Adding **depth** can improve predictive performance for both kernels and CNNs.
- Our deep convolutional kernel provides superior performance to kernels constructed using neural network gradient dynamics (NTK)
- Incorporating efficient data augmentation and scaling to large datasets is an open problem for convolutional kernels.



Code is available at:

https://github.com/modestyachts/neural_kernels_code